Senior Project Defense

Table of Contents

Introduction

Contact Information

Michael C. Wilcome
1875 North Smokerise Way,
Mt. Pleasant, SC, 29466
Mobile: (843) 276-8508
Fps.Genkei@gmail.com

Under the Supervision of

Gary Underwood
Professor of Computer Science/Advisor
Charleston Southern University
gunderwood@csuniv.edu

School Information

Charleston Southern University
9200, University Blvd. North Charleston, SC, 29406
(843) 863-7000

Introduction

Purpose and Background Information

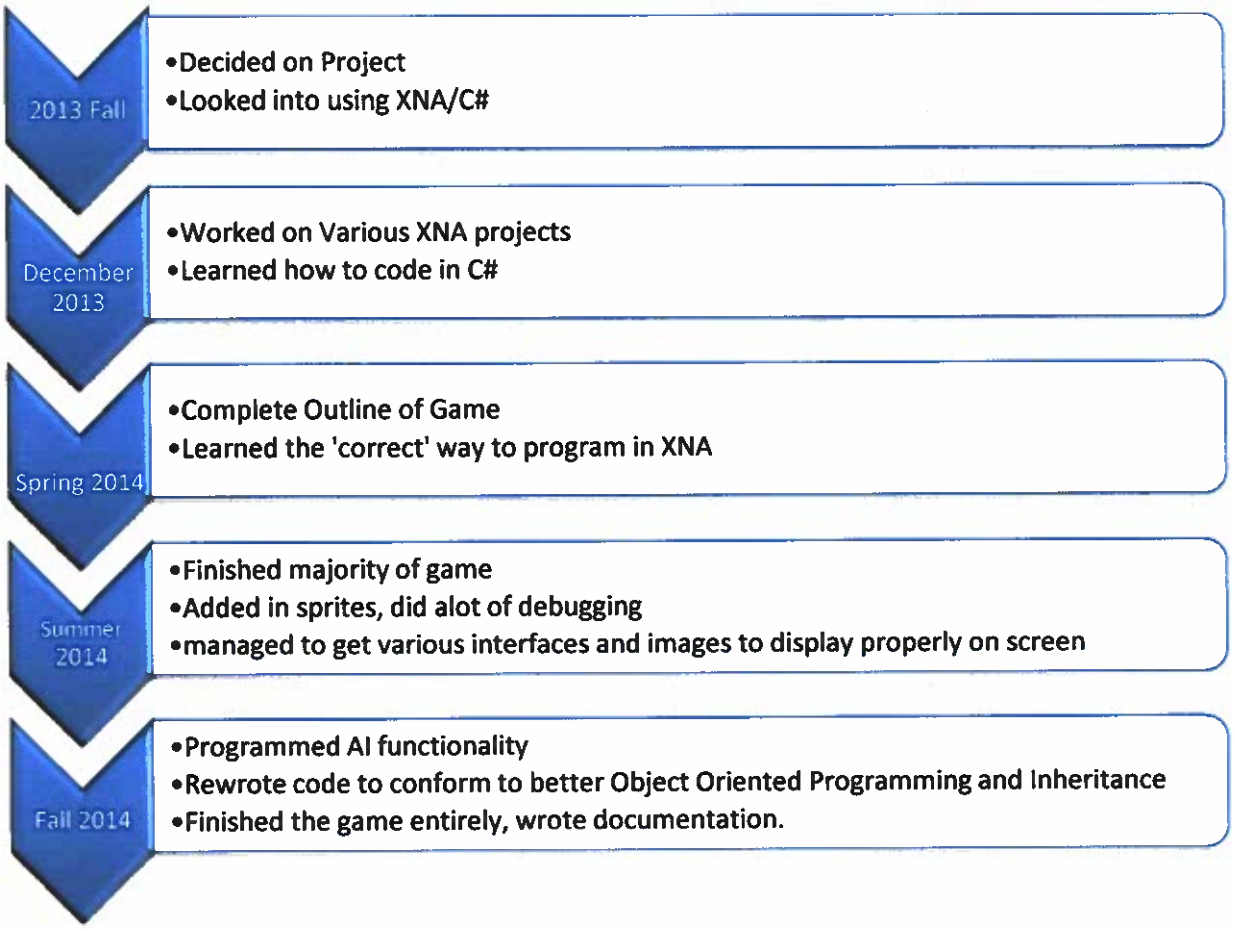
The purpose of this project was to fulfill the goal I set for myself when I began college. When I began attending Charleston Southern, it was my dream to become a game programmer. I wanted to graduate with the knowledge required to make a game. After five years at this university, I have found that I enjoy much more than just games in the computing world, and find the intricacies of programming fascinating. As I have developed, I have learned many concepts necessary to be a strong programmer, and I have used that knowledge to create this project.

In order to make this game, I used the ideas of another game I had seen called "Gol'Crop" by a company named Redshift. I played this game in my spare time and spent many hours contemplating what it would take to recreate it. I decided to use a windows platform because that was most readily available to me. Having already programmed in C++, I learned that there was an extension for visual studio called "XNA Game Studio" by Microsoft Corporation. This uses a language called C# and helps give programmers a platform to create games.

I began this project in fall of 2013. Under the advisement of Gary Underwood, we decided this game was worthy of being a senior level project. I had outlined my entire project from start to finish during the fall semester time period, and planned to begin work on it in December. It took many hours of finding tutorials and various projects in XNA before I managed to get an actual project working. During the spring of 2014, I managed to finally begin to figure out how XNA worked, and outline exactly how this game was to be recreated. I found out more about what I could not do than what I could do during this time period. Once the summer of 2014 began, I was able to spend most of my free time on this project, and therefore was able to finally figure out how everything worked. I began programming and finished the majority of work that needed to be done, such as getting cards into the game, and having various game screens. By the time the summer ended, I was able to debug the program better, as it would actually

display graphics, and allow events to occur properly. I had spent the majority of my time adding in card graphics, and custom making every card attribute so that it fit with my particular game. At this time, I saw my advisor to get a better opinion on the matter of how I needed to continue. At this point I was advised to make the game have better inheritance. I had not at the time created a "player" class, and everything was handled with global variables. This was considered not good programming. Therefore, throughout the fall semester I reworked my program to include classes, and extra functions so that I could reuse my code. Instead of retyping hundreds of lines, I managed to reduce my code significantly by adding in special functionality that will be explained later in the Program Details section.

Below is a basic timeline of the project, from start to finish:



How to Play

Deck Types

There are three different decks in this game that are currently playable:

The first deck is called the "Warrior" deck. This deck consists of mainly cards that either block damage, or heal. It does have cards that can deal damage, including cards that cause damage per turn, and cards that cause an initial amount of damage. It is considered the hardest deck to play as because of its lack of versatility and lack of much damage. It is best to play this deck by capitalizing on the healing and blocking while slowly taking down your opponents health.

The second deck is a little easier to play as, and is called the "Sorcerer" deck. It consists of cards that have utility and do lots of damage. The utility cards generally block damage and deal small amounts of damage each turn, or destroy the enemies markers. Being able to destroy markers at the correct time is a big advantage to the Sorcerer, as it can cause a normally bad situation (when the enemy has many good markers on the field) to become a much better one.

The third and final deck is called the "Summoner." This deck capitalizes on three main types of very powerful cards. The first are "minions" which the sorcerer can use to both block damage and cause damage. Some even steal life from the opponent. Another type of card is one that deals a small amount of damage to the summoner in order to deal much greater damage to the opponent. For example, there are cards which can deal 5 damage to the opponent and 1 to the summoner. The last type of card is the life stealing card. They allow the summoner to take a certain amount of life from the opponent and heal their own life with it. With all of this, it makes the Summoner a force to be reckoned with.

The reason why some decks are considered harder to play or win with than others is because some people like more of a challenge. The original game intended each deck to have an advantage over the next, and therefore that is how they are set up in this version of the game.

Cards

The various decks all use different types of cards, but they do have many similarities. Therefore, although they are different, they can be placed in a few main categories:

Cards that are instant

These cards can either deal damage, heal you, or some combination of both. For example, a card might just do "5 damage to opponent" or it might deal damage to both you and your opponent. Some will both heal you and damage the opponent. Finally, others may just be a flat heal amount.

Cards that are temporary

These cards usually create what is called a "marker." You may have 3 of these cards in play at any one time. If you play one while your markers are full, it will delete one to make room. Two sub-categories are formed from this type of card. One is a "minion" type card, which has its own health and will take damage for the one who 'summoned' it until it is destroyed. The second type of marker lasts a certain amount of turns and has no health, so it can only be destroyed by "destroy marker" cards.

Game Overview

The game is played with a human player and an AI. Both start at maximum of 40 health. The object of the game is to make the opponents health get to zero before yours does. You achieve this by playing cards that damage the other player until they surrender or lose. In order to choose a card, simply click the card in your hand, and then click the deck to end your turn. This begins the AI's turn, where they will make some intelligent decision, based on the level you chose them to be, and when they are finished, it will be the players turn again. To see which decks or which level of AI to play against, see the sections about decks and cards for more information.

Program Details

Challenges

There were many challenges I faced as I created this project. There are over ten different projects that were created and then thrown away before coming up with this final product. The reasons are mainly because of the way that XNA works, my lack of knowledge about it before taking on the project, and also the way that games are created in general (with various screens, etc...)

The first challenge I ran across was how to manage different screens. XNA does not come with a built in way to go from one screen to the next, so it was necessary for me to look up different ways to do this. For a while I considered using something built by Microsoft called a "Screen Manager." This essentially keeps track of each screen and allows a user to go back and forth from them. Unfortunately I felt it was very difficult to add my own code to this pre-built project. It looked impressive, but it used over 20 different files to implement the screen, and it was hard to sift through. I didn't quite figure out how to fully implement different screens until midway through summer of 2014. Eventually I found out that it was as simple as creating an "enum" variable which allowed me to sift through the different screens within the "update" method, which was initially given as part of XNA.

This leads to the next challenge I faced (although many of these were worked on simultaneously). This challenge was figuring out what XNA gave me, and how each of the methods worked. These will be explained further in the "How XNA Works" section.

Some other challenges included placing images over the various screens, creating cards objects, making player and AI a separate class, and reducing much of the code through reusable functions.

Various Algorithms

There are a few algorithms that I had to create to allow the game to run smoothly and efficiently. Among these are shuffling algorithms, AI algorithms, and the functions for actually playing cards. This also includes the Update and Draw methods which were given with XNA.

The first algorithm that I created was an algorithm to shuffle cards. It's based off of an already existing algorithm called the "Fisher-Yates" method. It essentially picks two cards using the built in "random" function, and swaps them. It does this for each pair of cards that exists in the deck, until all have been swapped.

The second algorithm I created was by far the hardest to figure out. It is my "PlayerPlayCard" function. The reason this took so long to implement was because at first, I was not using inheritance properly. I had originally attempted to have two separate functions so that one could be used by the AI and one could be used by the Player. The reason it needed to be two different functions was because player and AI were not their own class, they were not even objects. They were just a deck array, a card array, and a marker array. They also had some global variables associated like "health" but they weren't true objects. This was a very poor implementation because I was not able to treat them like "objects." I spent the majority of the fall 2014 semester fixing this problem. I did this by making a "Player" class which had all of the necessary variables, and created player objects for AI and Human. This way I was able to create only one PlayerPlayCard function that takes in a card, and two players. The card is the card that the player clicked or the AI chose (AI function will be discussed later). The first player given is the player playing the card, and the second player is the player that the card is acting on. This means that we can say "during humans turn, human clicked this card" and give it the card clicked, human, then AI. The function then goes through a number of checks to make sure the proper values are changed in this manner:

> First, it deals with the current card. It takes the card out of the "players" hand (could be AI or Human) and places it in the "current card" slot. It then checks if this card is temporary or minion, and if it is, it calls another function that deals with that called "SeeIfMinion." This method essentially looks at this marker in play, and compares it with other markers or the other player properly. The current card is placed in the markers section and it

deals damage or loses a turn accordingly. If the current card is not a minion or temporary, then we move onto the next portion, which just deals with the markers in play. This is because each turn, some markers will do damage or heal, etc... So we need to take these into account. After it does everything necessary with markers (looping through and making sure each does what it should) it continues on to check for a few more scenarios. The next thing it checks for is "is it a normal card?" if it is, then we just play the card by doing damage to a minion or opposite player, or healing the player playing the card accordingly. It then checks if it ignores markers, in which case it only does damage or heals, regardless of what markers are on the field (some cards will attack an opponent directly). Finally it checks if the card actually removes markers. If it does, it sees how many it is supposed to remove and does so.

Notice that by making this function reusable, while using other functions (like the one that checks for minion cards) saves quite a lot of code, while keeping it organized. This is one of the main goals of object oriented programming, which allows the programmer to do this efficiently.

Use of Objects

Objects in this program are created from one of a few different classes. There are 3 that are currently implemented. The first is the "Sprite" class. These objects needed to be made separate because XNA does not define "sprites" which are basically images pasted at given positions by default. In order to put images on the screen it was necessary to make a "sprite" class which allows an object that has an image, and position associated with it. I made sprites as part of cards. Which is the second mentionable object. A Card is created by one function called the "setAll" function. It needs to define each card, and it does so with one very large switch statement. Each card has quite unique attributes, and therefore has to have each part of it separate. Although it might have been possible to create different types of cards through polymorphism, it was better to make one switch, because it actually ended up reducing the total amount of code by not having to include extra sub-classes. Cards are created and placed into a "deck" which is part of a "player" object. Two players are created in this game, an AI and a Human. These players have a list

deck, hand, and markers. They have health, and the AI has AI functions that it uses to determine which card should be played at what time.

How XNA Works

XNA Game Studio is an add-on for Visual Studio that allows programmers to create games in C#. It comes with a few basic methods which allow the programmer to efficiently set up a game the way they want to.

The game can be played on Windows Phone or a Microsoft PC. We are given an initial "main" class which calls something called "Game1." This is where all of the true main functions lie. Game1 begins by loading all of the necessary content, this "LoadContent" function includes the following:

- Sprite Images
- Background Images
- Sounds
- Rectangles (for defining areas)
- Something called a "Spritebatch" which allows various screens to be drawn
- Basic inputs (in this case, the state of the mouse)

After the LoadContent method is the UnloadContent method. This is called when the game exits. It does nothing in this project, however it could be made to output to a text file, save certain game attributes, etc whenever the game exits.

Finally the programmer is given two methods that work together to produce all of what actually occurs in the game. They are called "Update" and "Draw." These functions are called many times every second of real-time.

Update is first called. In update, we swap the enum variable which allows us to switch from one screen to another. We also wait for user input and change values appropriately. This method calls all of the functions that make the game actually run underneath. In this project, Update does the following:

- Gets the current mouse state
- uses a switch-case to tell which state the game is in. It begins with the StartMenu
- It checks to see if the user is either hovering over or clicked something

- Once clicked it switches states
-After the "StartMenu" is finished, it calls the "AILevel" menu, which sets the level of AI for the human player depending on their choice
- Once the AI level is picked, the game begins. This is where Update will check if a card is clicked, then the deck is clicked, so that the player can end turn, the AI can begin turn, then afterwards it resets back to the players turn again
- Finally, it always checks in the main game screen whether one of the players health is zero. If they are, it exits the game.
- If at any time the player wishes to forfeit, they can click the "X" to quit the game as well.

Finally we have Draw. This uses all of the values changed by Update to actually draw objects on the screen. So, if Update changes the Gamestate, the Draw method will know and update the screen visually. Whenever Update changes a value like a players health, the Draw method will display the health value each iteration as well. So, about 60 times every second update is called, then draw is called. This is called the heartbeat or game loop. In the future, it would be possible to change a screen so that perhaps a "win" or "lose" screen appears after the end of the game. It could then track wins/losses and display them accordingly. It might even be able to ask if the player wants to play again. Given more time, this project could implement that.

AI Explained

The AI functions are a separate section because they require explaining separate from the algorithms. This section details the various processes that the AI go through at each level, and why this functionality was chosen over others.

The level 1 AI chooses a Warrior deck, which is considered the easiest to beat by the original game. It then only plays the first card in its hand. This is not so much "intelligent" as it is artificial. It is random enough so that it can be challenging, but it is not meant to be hard for the player to beat. This is the most simple AI by far.

The level 2 AI chooses a Sorcerer deck. This function takes into account how many markers the human player has in play, as well as the humans health. It always will attempt to win if the human has less health than its highest damage

card. If this AI has less than 10 health, it will usually heal (again, unless it can win). If it's health isn't less than 10 or it cannot win in one turn, then it chooses it's strength as the best option next. If it has a card that can destroy markers, AND the human has 2 or more markers, then it will attempt to play that card. Otherwise it will pick the card that deals the most damage. This proves to be a fairly intelligent AI, that bases its choice on the situation.

The level 3 AI picks a Summoner deck. It takes the level 2 AI a little further and tailors it to the Summoner decks strength. It looks through its hand attempting to find a minion card. It will choose a minion over a card that deals the most damage, UNLESS it can win automatically with that card. It knows which cards are minions in its hand at this point, but it will also check for cards that heal. If a card has a particularly high healing value, and the AI's health is reasonably low, it will attempt to heal instead of using the minion (though it keeps in mind the minion choice, depending on the minions health as well). So if the Summoner has less than 35 health, it will play a minion, otherwise it will attempt to play the card with the highest damage. If no card exists with a damage value, it will play the last card in its hand. We want to save minions for medium health situations, and save heal cards for low health situations. The high damage cards we attempt to either use right away or when the human has low enough health to lose from it. As the summoner cannot destroy markers, it will not attempt to look for cards that destroy markers.

Screenshots and Code

Program.cs

```
using System;

namespace SeniorProject
{
#if WINDOWS || XBOX
    static class Program
    {
        /// <summary>
        /// The main entry point for the application.
        /// </summary>
        static void Main(string[] args)
        {
            using (Game1 game = new Game1())
            {
                game.Run();
            }
        }
    }
#endif
}
```

Sprite.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Content;
using Microsoft.Xna.Framework.Graphics;

namespace SeniorProject
{
    class Sprite
    {
        //The current position of the Sprite
        public Vector2 Position = new Vector2(0, 0);

        //The texture object used when drawing the sprite
        private Texture2D mSpriteTexture;

        //Load the texture for the sprite using the Content Pipeline
        public void LoadContent(ContentManager theContentManager, string theAssetName)
        {
            mSpriteTexture = theContentManager.Load<Texture2D>(theAssetName);
        }
```

```
        //Draw the sprite to the screen
        public void Draw(SpriteBatch theSpriteBatch)
        {
            theSpriteBatch.Draw(mSpriteTexture, Position, Color.White);
        }
    }
}
```

Card.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace SeniorProject
{
     public class Card
    {
        /// <summary>
        /// Meat of the cards. This is everything that defines "card" from the graphic to
the
        /// set/get to the description, and everything inbetween.
        /// </summary>

        //Attributes of Card
        int DamageMe;
        string Description;
        int CardValue;
        int DamageThem;
        int Heal;
        string SpriteName;
        bool IsTemporary;
        bool IsMinion;
        bool RemovesMarker;
        bool DisablesMarker;
        int MarkersRemoved;
        int Turns;
        int MinionHealth;
        bool IsNormal;
        bool HasTurns;

        public Card()
        {
            //Set default values for the cards
            DamageMe = 0;
            Description = "This Card does not contain a \n description.";
            CardValue = 0;
            DamageThem = 0;
            Heal = 0;
            SpriteName = " ";
            IsTemporary = false;
            IsMinion = false;
            RemovesMarker = false;
```

```
            DisablesMarker = false; //used for "ignore defense" type of cards.
            IsNormal = false;
            MarkersRemoved = 0;
            Turns = 0;
            MinionHealth = 0;
            HasTurns = false;
            //IsBlock seems possible, with integer truncation, but it requires extra math
later which is:
            //Multiply Damage by Block so 7 inc damage * 30% block is 7 * 3, then divide
by 10 and truncate.
            //It makes it 21/10 = 2.1 truncated down is 2 damage blocked, so inc damage =
7-2 = 5.
            //Do this because I want ONLY integers, no Floating Points which is what
happens when you multiply.

        }

        #region Set Methods
        //Only one, it's called "set all" and creates the card based on the card value.

         public void setAll(int cv, int decktype)
         {
            /* This is going to take up most of the code, it sets every card based on its
card value.
             * I already know the attributes I want to give to the cards, so it's just a
matter of
             * creating them. I will handle each deck separately, so given the card value
and deck,
             * I use a switch to create the card.*/

            if (decktype == 1) //This will create any card if it is a Summoner* deck
which was chosen.
            {
                switch (cv)
                {
                    case 1:
                        Description = "Inflicts 1 damage per turn \n Turns: 3";
                        DamageThem = 1;
                        CardValue = 1;
                        Turns = 3;
                        HasTurns = true;
                        IsTemporary = true;
                        SpriteName = "AcidRain";
                        break;
                    case 2:
                        Description = "Inflicts 2 damage to opponent.";
                        DamageThem = 2;
                        CardValue = 2;
                        IsNormal = true;
                        SpriteName = "Atrophy";
                        break;
                    case 3:
                        Description = "Drains 12hp from the enemy.";
                        DamageThem = 12;
                        Heal = 12;
                        CardValue = 3;
                        IsNormal = true;
                        SpriteName = "Blackout";
```

```
            break;
        case 4:
            Description = "Inflicts 10hp damage per turn \n - Turns: 2";
            DamageThem = 10;
            IsTemporary = true;
            Turns = 2;
            HasTurns = true;
            CardValue = 4;
            SpriteName = "Disease";
            break;
        case 5:
            Description = "Inflicts 4 damage for suffering 2.";
            DamageThem = 4;
            DamageMe = 2;
            CardValue = 5;
            IsNormal = true;
            SpriteName = "SoulPower";
            break;
        case 6:
            Description = "Inflicts 5 damage for suffering 1.";
            DamageThem = 5;
            DamageMe = 1;
            CardValue = 6;
            IsNormal = true;
            SpriteName = "LifeBlood";
            break;
        case 7:
            Description = "Minion, inflicts 1 damage per \n turn. Hp:6";
            IsMinion = true;
            DamageThem = 1;
            MinionHealth = 6;
            CardValue = 7;
            SpriteName = "Golem";
            break;
        case 8:
            Description = "Inflicts 6 for suffering 3.";
            IsNormal = true;
            DamageThem = 6;
            DamageMe = 3;
            CardValue = 8;
            SpriteName = "Fang";
            break;
        case 9:
            Description = "Inflicts 5 for suffering 3.";
            IsNormal = true;
            DamageThem = 5;
            DamageMe = 3;
            CardValue = 9;
            SpriteName = "Sacrifice";
            break;
        case 10:
            Description = "Drains 3 Hp from enemy";
            IsNormal = true;
            DamageThem = 3;
            Heal = 3;
            CardValue = 10;
            SpriteName = "DarkTouch";
            break;
```

```
                case 11:
                    Description = "Inflicts 4 for suffering 1";
                    IsNormal = true;
                    DamageThem = 4;
                    DamageMe = 1;
                    CardValue = 11;
                    SpriteName = "PoisonCloud";
                    break;
                case 12:
                    Description = "Drains 6 Hp from enemy";
                    IsNormal = true;
                    DamageThem = 6;
                    Heal = 6;
                    CardValue = 12;
                    SpriteName = "VampireBite";
                    break;
                case 13:
                    Description = "Inflicts 15 damage for \n suffering 5";
                    IsNormal = true;
                    DamageThem = 15;
                    DamageMe = 5;
                    CardValue = 13;
                    SpriteName = "SoulCrush";
                    break;
                case 14:
                    Description = "Defensive Bone Wall. \n Hp: 6";
                    IsMinion = true;
                    MinionHealth = 6;
                    CardValue = 14;
                    SpriteName = "BoneWall";
                    break;
                case 15:
                    Description = "Defensive Bone Wall. \n Hp: 8";
                    IsMinion = true;
                    MinionHealth = 8;
                    CardValue = 15;
                    SpriteName = "RedBoneWall";
                    break;
                case 16:
                    Description = "Minion, inflicts 2 damage to \n enemy each turn.
Hp: 4";
                    IsMinion = true;
                    MinionHealth = 4;
                    DamageThem = 2;
                    CardValue = 16;
                    SpriteName = "Ghoul";
                    break;
                case 17:
                    Description = "Regenerates 1 Hp per \n turn. Turns: 5";
                    IsTemporary = true;
                    HasTurns = true;
                    Turns = 5;
                    Heal = 1;
                    CardValue = 17;
                    SpriteName = "Graveyard";
                    break;
                case 18:
                    Description = "Minion, inflicts 3 damage \n each turn. Hp:4";
```

```
            IsMinion = true;
            MinionHealth = 4;
            DamageThem = 3;
            CardValue = 18;
            SpriteName = "Skeleton";
            break;
        case 19:
            Description = "Minion, drains 3Hp in \n each turn. Turns:3";
            IsTemporary = true;
            Turns = 3;
            HasTurns = true;
            DamageThem = 3;
            Heal = 3;
            CardValue = 19;
            SpriteName = "Vampire";
            break;
        case 20:
            Description = "Drains 1Hp per turn. \n Turns: 2";
            IsTemporary = true;
            HasTurns = true;
            Turns = 2;
            DamageThem = 1;
            Heal = 1;
            CardValue = 20;
            SpriteName = "Parasite";
            break;
        case 21:
            Description = "Inflicts 2 damage per turn. \n Turns: 2";
            IsTemporary = true;
            HasTurns = true;
            Turns = 2;
            DamageThem = 2;
            CardValue = 21;
            SpriteName = "Zombie";
            break;
        case 22:
            Description = "Drains 1Hp per turn. \n Turns: 4";
            IsTemporary = true;
            HasTurns = true;
            Turns = 4;
            DamageThem = 1;
            Heal = 1;
            CardValue = 22;
            SpriteName = "Bats";
            break;
        case 23:
            Description = "Drains 1 Hp per turn. \n Turns: 3";
            IsTemporary = true;
            HasTurns = true;
            Turns = 3;
            DamageThem = 1;
            Heal = 1;
            CardValue = 23;
            SpriteName = "RedSeal";
            break;
        default:
            break;
    }
```

```
            }
            else if (decktype == 2) //This will create any card given that the deck was
Sorcerer*
            {
                switch (cv)
                {
                    case 1:
                        Description = "Air Aura: blocks 5 damage \n total.";
                        IsMinion = true;
                        MinionHealth = 5;
                        CardValue = 1;
                        SpriteName = "AirAura"; //when image is LOADED you look in
SorcererCards/AirAura, etc...
                        //Example: AirAura =
Content.Load<Texture2D>("SorcererCards/AirAura");
                        //         spriteBatch.Draw(AirAura, Vector2.blah, Color.White);
                        break;
                    case 2:
                        Description = "Earth Aura: blocks 7 damage \n total.";
                        IsMinion = true;
                        MinionHealth = 7;
                        CardValue = 2;
                        SpriteName = "EarthAura";
                        break;
                    case 3:
                        Description = "Fire Aura: blocks 3 and does 1 \n damage each turn
for 3 turns";
                        IsMinion = true;
                        MinionHealth = 3;
                        CardValue = 3;
                        Turns = 3;
                        HasTurns = true;
                        SpriteName = "FireAura";
                        break;
                    case 4:
                        Description = "Water Aura: blocks 5 damage \n total.";
                        IsMinion = true;
                        MinionHealth = 5;
                        CardValue = 4;
                        SpriteName = "WaterAura";
                        break;
                    case 5:
                        Description = "Inflicts 3 damage per turn \n Turns: 2";
                        IsTemporary = true;
                        DamageThem = 3;
                        Turns = 2;
                        HasTurns = true;
                        CardValue = 5;
                        SpriteName = "BladeStorm";
                        break;
                    case 6:
                        Description = "Heals 15 points.";
                        Heal = 15;
                        CardValue = 6;
                        IsNormal = true;
                        SpriteName = "Bless";
                        break;
                    case 7:
```

```
                Description = "Deals 1 damage per turn \n Turns: 6";
                DamageThem = 1;
                IsTemporary = true;
                Turns = 6;
                HasTurns = true;
                CardValue = 7;
                SpriteName = "Blizzard";
                break;
            case 8:
                Description = "Regenerates Hp by 2 per \n turn. Turns: 10";
                IsTemporary = true;
                Turns = 10;
                HasTurns = true;
                CardValue = 8;
                Heal = 2;
                SpriteName = "LifeNode";
                break;
            case 9:
                Description = "Deals 2 damage per turn. \n Turns: 8";
                IsTemporary = true;
                Turns = 8;
                HasTurns = true;
                DamageThem = 2;
                CardValue = 9;
                SpriteName = "DeathNode";
                break;
            case 10:
                Description = "Deals 1 damage and heals 1 \n damage per turn.
Turns: 4";
                IsTemporary = true;
                Turns = 4;
                HasTurns = true;
                DamageThem = 1;
                Heal = 1;
                CardValue = 10;
                SpriteName = "MoonNode";
                break;
            case 11:
                Description = "Deals 4 damage per turn. \n Turns: 4";
                IsTemporary = true;
                Turns = 4;
                HasTurns = true;
                DamageThem = 4;
                CardValue = 11;
                SpriteName = "SunNode";
                break;
            case 12:
                Description = "Deals 4 damage to enemy";
                DamageThem = 4;
                IsNormal = true;
                CardValue = 12;
                SpriteName = "IceBolt";
                break;
            case 13:
                Description = "Deals 5 damage to enemy";
                DamageThem = 5;
                IsNormal = true;
                CardValue = 13;
```

```
        SpriteName = "ChainLightning";
        break;
    case 14:
        Description = "Deals 5 damage to enemy";
        DamageThem = 5;
        IsNormal = true;
        CardValue = 14;
        SpriteName = "Earthquake";
        break;
    case 15:
        Description = "Deals 5 damage to enemy \n and destroys a marker";
        DamageThem = 5;
        MarkersRemoved = 1;
        RemovesMarker = true;
        IsNormal = true;
        CardValue = 15;
        SpriteName = "Whirlwind";
        break;
    case 16:
        Description = "Inflicts 8 damage to enemy";
        IsNormal = true;
        DamageThem = 8;
        CardValue = 16;
        SpriteName = "Firewall";
        break;
    case 17:
        Description = "Destroys all enemy markers";
        RemovesMarker = true;
        MarkersRemoved = 3;
        CardValue = 17;
        SpriteName = "Starfall";
        break;
    case 18:
        Description = "Inflicts 1 damage per turn. \n Turns: 5";
        DamageThem = 1;
        HasTurns = true;
        IsTemporary = true;
        Turns = 5;
        CardValue = 18;
        SpriteName = "Sandstorm";
        break;
    case 19:
        Description = "Heals 2 per turn. Turns: 2";
        IsTemporary = true;
        HasTurns = true;
        Turns = 3;
        Heal = 2;
        CardValue = 19;
        SpriteName = "Willowisp";
        break;
    case 20:
        Description = "Inflicts 2 damage per turn. \n Turns: 4";
        HasTurns = true;
        Turns = 4;
        DamageThem = 2;
        IsTemporary = true;
        CardValue = 20;
        SpriteName = "Wildfire";
```

```
                    break;
                case 21:
                    Description = "Deals 4 damage per turn. \n Turns: 3";
                    HasTurns = true;
                    Turns = 3;
                    DamageThem = 4;
                    IsTemporary = true;
                    CardValue = 21;
                    SpriteName = "RageofNature";
                    break;
                case 22:
                    Description = "Heals 4 points.";
                    Heal = 4;
                    IsNormal = true;
                    CardValue = 22;
                    SpriteName = "HealingShower";
                    break;
                case 23:
                    Description = "Heals 7 points.";
                    Heal = 7;
                    IsNormal = true;
                    CardValue = 23;
                    SpriteName = "HealingScroll";
                    break;
                default:
                    break;
            }
        }
        else //decktype will have to == 3, because I had to send SOME number to this
method. Will create Warrior*
        {
            switch (cv)
            {
                case 1:
                    Description = "Inflicts 2 damage to enemy.";
                    DamageThem = 2;
                    CardValue = 1;
                    IsNormal = true;
                    SpriteName = "Slash";
                    break;
                case 2:
                    Description = "Inflicts 3 damage to enemy.";
                    DamageThem = 3;
                    CardValue = 2;
                    IsNormal = true;
                    SpriteName = "Crush";
                    break;
                case 3:
                    Description = "Inflicts 4 damage to enemy.";
                    DamageThem = 4;
                    CardValue = 3;
                    IsNormal = true;
                    SpriteName = "RapidStab";
                    break;
                case 4:
                    Description = "Enemy bleeds 3hp for 3 turns.";
                    DamageThem = 3;
                    IsTemporary = true;
```

```
                Turns = 3;
                HasTurns = true;
                CardValue = 4;
                SpriteName = "Bleeding";
                break;
            case 5:
                Description = "Deals 2 damage and removes \n an enemy marker.";
                DamageThem = 2;
                RemovesMarker = true;
                IsNormal = true;
                MarkersRemoved = 1;
                CardValue = 5;
                SpriteName = "Assault";
                break;
            case 6:
                Description = "Deals heavy 8 damage to \n opponent.";
                DamageThem = 8;
                CardValue = 6;
                IsNormal = true;
                SpriteName = "DualStrike";
                break;
            case 7:
                Description = "Deals 5 damage and ignores \n enemy defenses this
turn.";
                DamageThem = 5;
                DisablesMarker = true;
                IsNormal = true;
                CardValue = 7;
                SpriteName = "BackStab";
                break;
            case 8:
                Description = "Heals 5 points per turn. \n Turns: 2";
                Heal = 5;
                IsNormal = true;
                HasTurns = true;
                Turns = 2;
                CardValue = 8;
                SpriteName = "Block";
                break;
            case 9:
                Description = "Shield protects you. \n Durability: 8 Hp";
                IsMinion = true;
                MinionHealth = 8;
                CardValue = 9;
                SpriteName = "TowerShield";
                break;
            case 10:
                Description = "2 damage per turn. \n Turns: 5";
                IsTemporary = true;
                DamageThem = 2;
                HasTurns = true;
                Turns = 5;
                CardValue = 10;
                SpriteName = "LongSword";
                break;
            case 11:
                Description = "Prevents damage with large shield. \n Durability:
12 Hp";
```

```
        IsMinion = true;
        MinionHealth = 12;
        CardValue = 11;
        SpriteName = "KiteShield";
        break;
    case 12:
        Description = "2 Damage per turn. \n Turns: 7";
        IsTemporary = true;
        HasTurns = true;
        Turns = 7;
        CardValue = 12;
        SpriteName = "BattleAxe";
        break;
    case 13:
        Description = "Plate Mail defends you. \n Durability 12 Hp.";
        IsMinion = true;
        MinionHealth = 12;
        CardValue = 13;
        SpriteName = "PlateMail";
        break;
    case 14:
        Description = "8 damage per turn. Turns: 2";
        IsTemporary = true;
        HasTurns = true;
        Turns = 2;
        DamageThem = 8;
        CardValue = 14;
        SpriteName = "Mourner";
        break;
    case 15:
        Description = "Heals 5 points";
        IsNormal = true;
        Heal = 5;
        CardValue = 15;
        SpriteName = "Remedy";
        break;
    case 16:
        Description = "Heals 3 per turn. Turns: 3";
        IsTemporary = true;
        HasTurns = true;
        Turns = 3;
        Heal = 3;
        CardValue = 16;
        SpriteName = "HealingPotion";
        break;
    case 17:
        Description = "Heals 12 points";
        IsNormal = true;
        Heal = 12;
        CardValue = 17;
        SpriteName = "Rejuvinate";
        break;
    case 18:
        Description = "Heals 10 points";
        IsNormal = true;
        Heal = 10;
        CardValue = 18;
        SpriteName = "HeroicRemedy";
```

```
                    break;
                case 19:
                    Description = "Inflicts 3 damage per turn \n Turns: 3";
                    IsTemporary = true;
                    HasTurns = true;
                    Turns = 3;
                    DamageThem = 3;
                    CardValue = 19;
                    SpriteName = "ArrowStorm";
                    break;
                case 20:
                    Description = "Removes marker from opponent";
                    RemovesMarker = true;
                    MarkersRemoved = 1;
                    CardValue = 20;
                    SpriteName = "Fosse";
                    break;
                case 21:
                    Description = "Deal 5 damage per turn. \n Turns: 3";
                    DamageThem = 5;
                    IsTemporary = true;
                    HasTurns = true;
                    Turns = 3;
                    CardValue = 21;
                    SpriteName = "Marksman";
                    break;
                case 22:
                    Description = "5 damage dealt per turn. \n However 3 damage is
taken. Turns: 2";
                    DamageThem = 5;
                    DamageMe = 3;
                    IsTemporary = true;
                    HasTurns = true;
                    Turns = 2;
                    CardValue = 22;
                    SpriteName = "Berzerk";
                    break;
                default:
                    break;
            }
        }

    }

    #endregion

    #region Get Methods
    //Allows me to get any of the information that was stored in the cards
attributes.
     public bool AllFalse()
     {
         if (IsTemporary == false && IsMinion == false && RemovesMarker == false &&
DisablesMarker == false)
             return true;
         else
             return false;
     }
```

```
public bool DoesRemoveMarker()
{
    return RemovesMarker;
}

public bool SeeIfTemp()
{
    return IsTemporary;
}

public bool SeeIfNormal()
{
    return IsNormal;
}

public bool SeeIfMinion()
{
    return IsMinion;
}

public bool SeeIfRemovesMarker()
{
    return RemovesMarker;
}

public bool SeeIfDisablesMarker()
{
    return DisablesMarker; //remember this is for penetrating defense
}

public int getMinionHealth()
{
    return MinionHealth;
}

public int getTurns()
{
    return Turns;
}

public int getMarkersRemoved()
{
    return MarkersRemoved;
}

public string getSpriteName()
{
    return SpriteName;
}

public int getHeal()
{
    return Heal;
}

public int getDamageThem()
{
    return DamageThem;
```

```
        }

        public string getDescription()
        {
            return Description;
        }

        public int getCardValue()
        {
            return CardValue;
        }

        public int getDamageMe()
        {
            return DamageMe;
        }

        public bool DoesDisableMarker()
        {
            return DisablesMarker;
        }
        #endregion

        #region Extra Methods
        public bool UsesTurns()
        {
            return HasTurns;
        }

        public void LoseTurn()
        {
           Turns = Turns - 1;
        }

        public void LoseMinionHealth(int damage)
        {
            MinionHealth = MinionHealth - damage;
        }
        #endregion
    }
}
```

Game1.cs

```
//ALL Content, besides Microsoft base content and that of the ideas taken from Redshift's
"Gol'Crop" belong to:
//Michael C. Wilcome
//2013-2014
//Senior Project

using System;
using System.Collections.Generic;
using System.Linq;
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Audio;
```

```
using Microsoft.Xna.Framework.Content;
using Microsoft.Xna.Framework.GamerServices;
using Microsoft.Xna.Framework.Graphics;
using Microsoft.Xna.Framework.Input;
using Microsoft.Xna.Framework.Media;

namespace SeniorProject
{
    public class Game1 : Microsoft.Xna.Framework.Game
    {
        #region Necessary Class Variables
        GraphicsDeviceManager graphics;
        SpriteBatch spriteBatch;
        SpriteBatch startBatch;
        SpriteBatch AILevelBatch;
        SpriteBatch EndGameBatch;

        //Added Sprites
        Sprite background;
        Sprite SorcererHover;
        Sprite SummonerHover;
        Sprite WarriorHover;
        Sprite StartMenu;

        Sprite AIBaseLevelScreen;
        Sprite AILevel1Screen;
        Sprite AILevel2Screen;
        Sprite AILevel3Screen;

        //CARDS
        Sprite DefaultCard;

        Sprite AirAura;
        Sprite BladeStorm;
        Sprite Bless;
        Sprite Blizzard;
        Sprite EarthAura;
        Sprite FireAura;
        Sprite WaterAura;
        Sprite ChainLightning;
        Sprite DeathNode;
        Sprite Earthquake;
        Sprite Firewall;
        Sprite HealingScroll;
        Sprite HealingShower;
        Sprite IceBolt;
        Sprite LifeNode;
        Sprite MoonNode;
        Sprite RageofNature;
        Sprite Sandstorm;
        Sprite Starfall;
        Sprite SunNode;
        Sprite Whirlwind;
        Sprite Wildfire;
        Sprite Willowisp;

        Sprite AcidRain;
        Sprite Atrophy;
```

```
Sprite Blackout;
Sprite Disease;
Sprite Golem;
Sprite LifeBlood;
Sprite SoulPower;
Sprite Bats;
Sprite BoneWall;
Sprite DarkTouch;
Sprite Fang;
Sprite Ghoul;
Sprite Graveyard;
Sprite Parasite;
Sprite PoisonCloud;
Sprite RedBoneWall;
Sprite RedSeal;
Sprite Sacrifice;
Sprite Skeleton;
Sprite SoulCrush;
Sprite Vampire;
Sprite VampireBite;
Sprite Zombie;

Sprite Assault;
Sprite BackStab;
Sprite Bleeding;
Sprite Crush;
Sprite DualStrike;
Sprite RapidStab;
Sprite Slash;
Sprite ArrowStorm;
Sprite BattleAxe;
Sprite Berzerk;
Sprite Block;
Sprite Bloodlust;
Sprite Fosse;
Sprite HealingPotion;
Sprite HeroicRemedy;
Sprite KiteShield;
Sprite LongSword;
Sprite Marksman;
Sprite Mourner;
Sprite PlateMail;
Sprite Raid;
Sprite Rejuvinate;
Sprite Remedy;
Sprite TowerShield;

//Markers
Sprite AirAuraMarker;
Sprite BladeStormMarker;
Sprite BlessMarker;
Sprite BlizzardMarker;
Sprite EarthAuraMarker;
Sprite FireAuraMarker;
Sprite WaterAuraMarker;
Sprite ChainLightningMarker;
Sprite DeathNodeMarker;
Sprite EarthquakeMarker;
```

```
Sprite FirewallMarker;
Sprite HealingScrollMarker;
Sprite HealingShowerMarker;
Sprite IceBoltMarker;
Sprite LifeNodeMarker;
Sprite MoonNodeMarker;
Sprite RageofNatureMarker;
Sprite SandstormMarker;
Sprite StarfallMarker;
Sprite SunNodeMarker;
Sprite WhirlwindMarker;
Sprite WildfireMarker;
Sprite WillowispMarker;

Sprite AcidRainMarker;
Sprite AtrophyMarker;
Sprite BlackoutMarker;
Sprite DiseaseMarker;
Sprite GolemMarker;
Sprite LifeBloodMarker;
Sprite SoulPowerMarker;
Sprite BatsMarker;
Sprite BoneWallMarker;
Sprite DarkTouchMarker;
Sprite FangMarker;
Sprite GhoulMarker;
Sprite GraveyardMarker;
Sprite ParasiteMarker;
Sprite PoisonCloudMarker;
Sprite RedBoneWallMarker;
Sprite RedSealMarker;
Sprite SacrificeMarker;
Sprite SkeletonMarker;
Sprite SoulCrushMarker;
Sprite VampireMarker;
Sprite VampireBiteMarker;
Sprite ZombieMarker;

Sprite AssaultMarker;
Sprite BackStabMarker;
Sprite BleedingMarker;
Sprite CrushMarker;
Sprite DualStrikeMarker;
Sprite RapidStabMarker;
Sprite SlashMarker;
Sprite ArrowStormMarker;
Sprite BattleAxeMarker;
Sprite BerzerkMarker;
Sprite BlockMarker;
Sprite BloodlustMarker;
Sprite FosseMarker;
Sprite HealingPotionMarker;
Sprite HeroicRemedyMarker;
Sprite KiteShieldMarker;
Sprite LongSwordMarker;
Sprite MarksmanMarker;
Sprite MournerMarker;
Sprite PlateMailMarker;
```

```
        Sprite RaidMarker;
        Sprite RejuvinateMarker;
        Sprite RemedyMarker;
        Sprite TowerShieldMarker;

        //Only necessary for drawing
        Vector2 PHandS1;
        Vector2 PHandS2;
        Vector2 PHandS3;
        Vector2 PHandS4;
        Vector2 PHandS5;
        Vector2 CCPosition;

        Sprite sp1;
        Sprite sp2;
        Sprite sp3;
        Sprite sp4;
        Sprite sp5;

        Sprite Curr;

        //Marker positions/sprites
        Sprite HM1;
        Sprite HM2;
        Sprite HM3;
        Sprite AIM1;
        Sprite AIM2;
        Sprite AIM3;

        Vector2 HM1pos;
        Vector2 HM2pos;
        Vector2 HM3pos;
        Vector2 AIM1pos;
        Vector2 AIM2pos;
        Vector2 AIM3pos;

        //Need to add booleans (Like light switches that turn on and off) for DRAW to
figure out
        //When to draw. So, Update will update, then draw will look and see the game has
updated and draw it.
        bool SorcHover;
        bool SumHover;
        bool WarHover;

        bool AI1Hover;
        bool AI2Hover;
        bool AI3Hover;

        //Once game starts, these are on when mouse hovers or clicks on a card
        bool Card1Hover;
        bool Card2Hover;
        bool Card3Hover;
        bool Card4Hover;
        bool Card5Hover;

        //for the text drawing
        SpriteFont Font1;
        SpriteFont Font2;
```

```
Vector2 DescriptionPos;
Vector2 PHealthVec;
Vector2 AIHealthVec;
string DescriptionText;
string DefaultDesText;

//For rectangle hovers
Rectangle Card1Area;
Rectangle Card2Area;
Rectangle Card3Area;
Rectangle Card4Area;
Rectangle Card5Area;

Rectangle warArea;
Rectangle sorArea;
Rectangle sumArea;

Rectangle L1Area;
Rectangle L2Area;
Rectangle L3Area;

Rectangle DeckRec;
Rectangle ExitRec;

Rectangle HM1Area;
Rectangle HM2Area;
Rectangle HM3Area;
Rectangle AIM1Area;
Rectangle AIM2Area;
Rectangle AIM3Area;

Rectangle CurrentCardRec;


//Adding Music
Song BackgroundMusic;
bool songstart = false;

//Switching GameStates for Update
enum GameState { _StartMenu, _AILevelPick, _GameStart, _GameEnd }
GameState state = GameState._StartMenu;

//Tell tell what the current card in play is
List<Card> CurrentCard = new List<Card>();

//Player objects
Player Human;
Player AI;

bool HumanTurn;

MouseState _currentMouseState;
MouseState _previousMouseState;
#endregion

//Game Entry Point
#region Game1()
public Game1()
```

```
{
    graphics = new GraphicsDeviceManager(this);
    Content.RootDirectory = "Content";

    this.IsMouseVisible = true;
}
#endregion

//Initialize (Override) Method
#region Initialize
protected override void Initialize()
{
    //Add onto this when adding more cards**

    //initialize drawing sprites
    background = new Sprite();
    SorcererHover = new Sprite();
    WarriorHover = new Sprite();
    SummonerHover = new Sprite();
    StartMenu = new Sprite();

    AIBaseLevelScreen = new Sprite();
    AILevel1Screen = new Sprite();
    AILevel2Screen = new Sprite();
    AILevel3Screen = new Sprite();

    //CARD Sprites
    DefaultCard = new Sprite();

    AirAura = new Sprite();
    BladeStorm = new Sprite();
    Bless = new Sprite();
    Blizzard = new Sprite();
    EarthAura = new Sprite();
    FireAura = new Sprite();
    WaterAura = new Sprite();
    ChainLightning = new Sprite();
    DeathNode = new Sprite();
    Earthquake = new Sprite();
    Firewall = new Sprite();
    HealingScroll = new Sprite();
    HealingShower = new Sprite();
    IceBolt = new Sprite();
    LifeNode = new Sprite();
    MoonNode = new Sprite();
    RageofNature = new Sprite();
    Sandstorm = new Sprite();
    Starfall = new Sprite();
    SunNode = new Sprite();
    Whirlwind = new Sprite();
    Wildfire = new Sprite();
    Willowisp = new Sprite();

    AcidRain = new Sprite();
    Atrophy = new Sprite();
    Blackout = new Sprite();
    Disease = new Sprite();
    Golem = new Sprite();
```

```
LifeBlood = new Sprite();
SoulPower = new Sprite();
Bats = new Sprite();
BoneWall = new Sprite();
DarkTouch = new Sprite();
Fang = new Sprite();
Ghoul = new Sprite();
Graveyard = new Sprite();
Parasite = new Sprite();
PoisonCloud = new Sprite();
RedBoneWall = new Sprite();
RedSeal = new Sprite();
Sacrifice = new Sprite();
Skeleton = new Sprite();
SoulCrush = new Sprite();
Vampire = new Sprite();
VampireBite = new Sprite();
Zombie = new Sprite();

Assault = new Sprite();
BackStab = new Sprite();
Bleeding = new Sprite();
Crush = new Sprite();
DualStrike = new Sprite();
RapidStab = new Sprite();
Slash = new Sprite();
ArrowStorm = new Sprite();
BattleAxe = new Sprite();
Berzerk = new Sprite();
Block = new Sprite();
Bloodlust = new Sprite();
Fosse = new Sprite();
HealingPotion = new Sprite();
HeroicRemedy = new Sprite();
KiteShield = new Sprite();
LongSword = new Sprite();
Marksman = new Sprite();
Mourner = new Sprite();
PlateMail = new Sprite();
Raid = new Sprite();
Rejuvinate = new Sprite();
Remedy = new Sprite();
TowerShield = new Sprite();

//Marker Sprites
AirAuraMarker = new Sprite();
BladeStormMarker = new Sprite();
BlessMarker = new Sprite();
BlizzardMarker = new Sprite();
EarthAuraMarker = new Sprite();
FireAuraMarker = new Sprite();
WaterAuraMarker = new Sprite();
ChainLightningMarker = new Sprite();
DeathNodeMarker = new Sprite();
EarthquakeMarker = new Sprite();
FirewallMarker = new Sprite();
HealingScrollMarker = new Sprite();
HealingShowerMarker = new Sprite();
```

```
IceBoltMarker = new Sprite();
LifeNodeMarker = new Sprite();
MoonNodeMarker = new Sprite();
RageofNatureMarker = new Sprite();
SandstormMarker = new Sprite();
StarfallMarker = new Sprite();
SunNodeMarker = new Sprite();
WhirlwindMarker = new Sprite();
WildfireMarker = new Sprite();
WillowispMarker = new Sprite();

AcidRainMarker = new Sprite();
AtrophyMarker = new Sprite();
BlackoutMarker = new Sprite();
DiseaseMarker = new Sprite();
GolemMarker = new Sprite();
LifeBloodMarker = new Sprite();
SoulPowerMarker = new Sprite();
BatsMarker = new Sprite();
BoneWallMarker = new Sprite();
DarkTouchMarker = new Sprite();
FangMarker = new Sprite();
GhoulMarker = new Sprite();
GraveyardMarker = new Sprite();
ParasiteMarker = new Sprite();
PoisonCloudMarker = new Sprite();
RedBoneWallMarker = new Sprite();
RedSealMarker = new Sprite();
SacrificeMarker = new Sprite();
SkeletonMarker = new Sprite();
SoulCrushMarker = new Sprite();
VampireMarker = new Sprite();
VampireBiteMarker = new Sprite();
ZombieMarker = new Sprite();

AssaultMarker = new Sprite();
BackStabMarker = new Sprite();
BleedingMarker = new Sprite();
CrushMarker = new Sprite();
DualStrikeMarker = new Sprite();
RapidStabMarker = new Sprite();
SlashMarker = new Sprite();
ArrowStormMarker = new Sprite();
BattleAxeMarker = new Sprite();
BerzerkMarker = new Sprite();
BlockMarker = new Sprite();
BloodlustMarker = new Sprite();
FosseMarker = new Sprite();
HealingPotionMarker = new Sprite();
HeroicRemedyMarker = new Sprite();
KiteShieldMarker = new Sprite();
LongSwordMarker = new Sprite();
MarksmanMarker = new Sprite();
MournerMarker = new Sprite();
PlateMailMarker = new Sprite();
RaidMarker = new Sprite();
RejuvinateMarker = new Sprite();
RemedyMarker = new Sprite();
```

```
TowerShieldMarker = new Sprite();

//Initializing player objects
Human = new Player();
AI = new Player();

DescriptionPos = new Vector2(375, 435);
PHealthVec = new Vector2(495, 308);
AIHealthVec = new Vector2(271, 174);

//Initializing new vectors/sprites for positioning cards on screen
PHandS1 = new Vector2(242, 341);
PHandS2 = new Vector2(303, 341);
PHandS3 = new Vector2(363, 341);
PHandS4 = new Vector2(422, 341);
PHandS5 = new Vector2(483, 341);
CCPosition = new Vector2(359, 173);

sp1 = new Sprite();
sp2 = new Sprite();
sp3 = new Sprite();
sp4 = new Sprite();
sp5 = new Sprite();
Curr = new Sprite();

//Card hover initialization
Card1Hover = false;
Card2Hover = false;
Card3Hover = false;
Card4Hover = false;
Card5Hover = false;

//Before game begins, deck is chosen & created
SorcHover = false;
SumHover = false;
WarHover = false;

//Then level is chosen
AI1Hover = false;
AI2Hover = false;
AI3Hover = false;

//Need rectangles to see if mouse is over a card or area
Card1Area = new Rectangle(243, 341, 47, 59);
Card2Area = new Rectangle(304, 341, 47, 59);
Card3Area = new Rectangle(364, 341, 47, 59);
Card4Area = new Rectangle(423, 341, 47, 59);
Card5Area = new Rectangle(484, 341, 47, 59);

warArea = new Rectangle(300, 95, 145, 65);
sorArea = new Rectangle(300, 190, 145, 65);
sumArea = new Rectangle(300, 290, 145, 65);

L1Area = new Rectangle(338, 178, 100, 34);
L2Area = new Rectangle(338, 224, 100, 34);
L3Area = new Rectangle(338, 270, 100, 34);

DeckRec = new Rectangle(267, 250, 54, 65);
```

```
        ExitRec = new Rectangle(227, 289, 33, 31);

        CurrentCardRec = new Rectangle(360, 172, 47, 59);

        HM1 = new Sprite();
        HM2 = new Sprite();
        HM3 = new Sprite();
        AIM1 = new Sprite();
        AIM2 = new Sprite();
        AIM3 = new Sprite();

        HM1pos = new Vector2(479, 214);
        HM2pos = new Vector2(448, 214);
        HM3pos = new Vector2(448, 246);
        AIM1pos = new Vector2(261, 82);
        AIM2pos = new Vector2(293, 82);
        AIM3pos = new Vector2(293, 113);

        HM1Area = new Rectangle(479, 214, 30, 30);
        HM2Area = new Rectangle(448, 214, 30, 30);
        HM3Area = new Rectangle(448, 246, 30, 30);
        AIM1Area = new Rectangle(261, 82, 30, 30);
        AIM2Area = new Rectangle(293, 82, 30, 30);
        AIM3Area = new Rectangle(293, 113, 30, 30);

        //to start
        HumanTurn = true;

        //Need to give default description display
        DescriptionText = "Card descriptions will be \n displayed here on hover.";
        DefaultDesText = "Card descriptions will be \n displayed here on hover.";

        base.Initialize();
    }
    #endregion

    //Basic Load Content Method
    #region Load Content
    protected override void LoadContent()
    {
        // Create a new SpriteBatch, which can be used to draw textures.
        spriteBatch = new SpriteBatch(GraphicsDevice);
        startBatch = new SpriteBatch(GraphicsDevice);
        AILevelBatch = new SpriteBatch(GraphicsDevice);
        EndGameBatch = new SpriteBatch(GraphicsDevice);
        //Can add "helpBatch" in future versions

        //loading Backgrounds
        background.LoadContent(this.Content, "background");
        background.Position = new Vector2(225, 0);

        StartMenu.LoadContent(this.Content, "StartMenu");
        StartMenu.Position = new Vector2(225, 0);

        SorcererHover.LoadContent(this.Content, "SorcererHover");
        SorcererHover.Position = new Vector2(225, 0);

        WarriorHover.LoadContent(this.Content, "WarriorHover");
```

```
WarriorHover.Position = new Vector2(225, 0);

SummonerHover.LoadContent(this.Content, "SummonerHover");
SummonerHover.Position = new Vector2(225, 0);

AIBaseLevelScreen.LoadContent(this.Content, "AIBaseLevelScreen");
AIBaseLevelScreen.Position = new Vector2(225, 0);

AILevel1Screen.LoadContent(this.Content, "AILevel1Screen");
AILevel1Screen.Position = new Vector2(225, 0);

AILevel2Screen.LoadContent(this.Content, "AILevel2Screen");
AILevel2Screen.Position = new Vector2(225, 0);

AILevel3Screen.LoadContent(this.Content, "AILevel3Screen");
AILevel3Screen.Position = new Vector2(225, 0);

//Load Cards, note that they won't have positions until update/draw
DefaultCard.LoadContent(this.Content, "DefaultCard");

AirAura.LoadContent(this.Content, "SorcererCards/AirAura");
BladeStorm.LoadContent(this.Content, "SorcererCards/BladeStorm");
Bless.LoadContent(this.Content, "SorcererCards/Bless");
Blizzard.LoadContent(this.Content, "SorcererCards/Blizzard");
EarthAura.LoadContent(this.Content, "SorcererCards/EarthAura");
FireAura.LoadContent(this.Content, "SorcererCards/FireAura");
WaterAura.LoadContent(this.Content, "SorcererCards/WaterAura");
ChainLightning.LoadContent(this.Content, "SorcererCards/ChainLightning");
DeathNode.LoadContent(this.Content, "SorcererCards/DeathNode");
Earthquake.LoadContent(this.Content, "SorcererCards/Earthquake");
Firewall.LoadContent(this.Content, "SorcererCards/Firewall");
HealingScroll.LoadContent(this.Content, "SorcererCards/HealingScroll");
HealingShower.LoadContent(this.Content, "SorcererCards/HealingShower");
IceBolt.LoadContent(this.Content, "SorcererCards/IceBolt");
LifeNode.LoadContent(this.Content, "SorcererCards/LifeNode");
MoonNode.LoadContent(this.Content, "SorcererCards/MoonNode");
RageofNature.LoadContent(this.Content, "SorcererCards/RageofNature");
Sandstorm.LoadContent(this.Content, "SorcererCards/Sandstorm");
Starfall.LoadContent(this.Content, "SorcererCards/Starfall");
SunNode.LoadContent(this.Content, "SorcererCards/SunNode");
Whirlwind.LoadContent(this.Content, "SorcererCards/Whirlwind");
Wildfire.LoadContent(this.Content, "SorcererCards/Wildfire");
Willowisp.LoadContent(this.Content, "SorcererCards/Willowisp");

AcidRain.LoadContent(this.Content, "SummonerCards/AcidRain");
Atrophy.LoadContent(this.Content, "SummonerCards/Atrophy");
Blackout.LoadContent(this.Content, "SummonerCards/Blackout");
Disease.LoadContent(this.Content, "SummonerCards/Disease");
Golem.LoadContent(this.Content, "SummonerCards/Golem");
LifeBlood.LoadContent(this.Content, "SummonerCards/LifeBlood");
SoulPower.LoadContent(this.Content, "SummonerCards/SoulPower");
Bats.LoadContent(this.Content, "SummonerCards/Bats");
BoneWall.LoadContent(this.Content, "SummonerCards/BoneWall");
DarkTouch.LoadContent(this.Content, "SummonerCards/DarkTouch");
Fang.LoadContent(this.Content, "SummonerCards/Fang");
Ghoul.LoadContent(this.Content, "SummonerCards/Ghoul");
Graveyard.LoadContent(this.Content, "SummonerCards/Graveyard");
Parasite.LoadContent(this.Content, "SummonerCards/Parasite");
```

```
            PoisonCloud.LoadContent(this.Content, "SummonerCards/PoisonCloud");
            RedBoneWall.LoadContent(this.Content, "SummonerCards/RedBoneWall");
            RedSeal.LoadContent(this.Content, "SummonerCards/RedSeal");
            Sacrifice.LoadContent(this.Content, "SummonerCards/Sacrifice");
            Skeleton.LoadContent(this.Content, "SummonerCards/Skeleton");
            SoulCrush.LoadContent(this.Content, "SummonerCards/SoulCrush");
            Vampire.LoadContent(this.Content, "SummonerCards/Vampire");
            VampireBite.LoadContent(this.Content, "SummonerCards/VampireBite");
            Zombie.LoadContent(this.Content, "SummonerCards/Zombie");

            Assault.LoadContent(this.Content, "WarriorCards/Assault");
            BackStab.LoadContent(this.Content, "WarriorCards/BackStab");
            Bleeding.LoadContent(this.Content, "WarriorCards/Bleeding");
            Crush.LoadContent(this.Content, "WarriorCards/Crush");
            DualStrike.LoadContent(this.Content, "WarriorCards/DualStrike");
            RapidStab.LoadContent(this.Content, "WarriorCards/RapidStab");
            ArrowStorm.LoadContent(this.Content, "WarriorCards/ArrowStorm");
            Slash.LoadContent(this.Content, "WarriorCards/Slash");
            BattleAxe.LoadContent(this.Content, "WarriorCards/BattleAxe");
            Berzerk.LoadContent(this.Content, "WarriorCards/Berzerk");
            Block.LoadContent(this.Content, "WarriorCards/Block");
            Bloodlust.LoadContent(this.Content, "WarriorCards/Bloodlust");
            Fosse.LoadContent(this.Content, "WarriorCards/Fosse");
            HealingPotion.LoadContent(this.Content, "WarriorCards/HealingPotion");
            HeroicRemedy.LoadContent(this.Content, "WarriorCards/HeroicRemedy");
            KiteShield.LoadContent(this.Content, "WarriorCards/KiteShield");
            LongSword.LoadContent(this.Content, "WarriorCards/LongSword");
            Marksman.LoadContent(this.Content, "WarriorCards/Marksman");
            Mourner.LoadContent(this.Content, "WarriorCards/Mourner");
            PlateMail.LoadContent(this.Content, "WarriorCards/PlateMail");
            Raid.LoadContent(this.Content, "WarriorCards/Raid");
            Rejuvinate.LoadContent(this.Content, "WarriorCards/Rejuvinate");
            Remedy.LoadContent(this.Content, "WarriorCards/Remedy");
            TowerShield.LoadContent(this.Content, "WarriorCards/TowerShield");

            //Load Markers
            AirAuraMarker.LoadContent(this.Content, "SorcererMarkers/AirAuraMarker");
            BladeStormMarker.LoadContent(this.Content,
"SorcererMarkers/BladeStormMarker");
            BlessMarker.LoadContent(this.Content, "SorcererMarkers/BlessMarker");
            BlizzardMarker.LoadContent(this.Content, "SorcererMarkers/BlizzardMarker");
            EarthAuraMarker.LoadContent(this.Content, "SorcererMarkers/EarthAuraMarker");
            FireAuraMarker.LoadContent(this.Content, "SorcererMarkers/FireAuraMarker");
            WaterAuraMarker.LoadContent(this.Content, "SorcererMarkers/WaterAuraMarker");
            ChainLightningMarker.LoadContent(this.Content,
"SorcererMarkers/ChainLightningMarker");
            DeathNodeMarker.LoadContent(this.Content, "SorcererMarkers/DeathNodeMarker");
            EarthquakeMarker.LoadContent(this.Content,
"SorcererMarkers/EarthquakeMarker");
            FirewallMarker.LoadContent(this.Content, "SorcererMarkers/FirewallMarker");
            HealingScrollMarker.LoadContent(this.Content,
"SorcererMarkers/HealingScrollMarker");
            HealingShowerMarker.LoadContent(this.Content,
"SorcererMarkers/HealingShowerMarker");
            IceBoltMarker.LoadContent(this.Content, "SorcererMarkers/IceBoltMarker");
            LifeNodeMarker.LoadContent(this.Content, "SorcererMarkers/LifeNodeMarker");
            MoonNodeMarker.LoadContent(this.Content, "SorcererMarkers/MoonNodeMarker");
```

```
            RageofNatureMarker.LoadContent(this.Content,
"SorcererMarkers/RageofNatureMarker");
            SandstormMarker.LoadContent(this.Content, "SorcererMarkers/SandstormMarker");
            StarfallMarker.LoadContent(this.Content, "SorcererMarkers/StarfallMarker");
            SunNodeMarker.LoadContent(this.Content, "SorcererMarkers/SunNodeMarker");
            WhirlwindMarker.LoadContent(this.Content, "SorcererMarkers/WhirlwindMarker");
            WildfireMarker.LoadContent(this.Content, "SorcererMarkers/WildfireMarker");
            WillowispMarker.LoadContent(this.Content, "SorcererMarkers/WillowispMarker");

            AcidRainMarker.LoadContent(this.Content, "SummonerMarkers/AcidRainMarker");
            AtrophyMarker.LoadContent(this.Content, "SummonerMarkers/AtrophyMarker");
            BlackoutMarker.LoadContent(this.Content, "SummonerMarkers/BlackoutMarker");
            DiseaseMarker.LoadContent(this.Content, "SummonerMarkers/DiseaseMarker");
            GolemMarker.LoadContent(this.Content, "SummonerMarkers/GolemMarker");
            LifeBloodMarker.LoadContent(this.Content, "SummonerMarkers/LifeBloodMarker");
            SoulPowerMarker.LoadContent(this.Content, "SummonerMarkers/SoulPowerMarker");
            BatsMarker.LoadContent(this.Content, "SummonerMarkers/BatsMarker");
            BoneWallMarker.LoadContent(this.Content, "SummonerMarkers/BoneWallMarker");
            DarkTouchMarker.LoadContent(this.Content, "SummonerMarkers/DarkTouchMarker");
            FangMarker.LoadContent(this.Content, "SummonerMarkers/FangMarker");
            GhoulMarker.LoadContent(this.Content, "SummonerMarkers/GhoulMarker");
            GraveyardMarker.LoadContent(this.Content, "SummonerMarkers/GraveyardMarker");
            ParasiteMarker.LoadContent(this.Content, "SummonerMarkers/ParasiteMarker");
            PoisonCloudMarker.LoadContent(this.Content,
"SummonerMarkers/PoisonCloudMarker");
            RedBoneWallMarker.LoadContent(this.Content,
"SummonerMarkers/RedBoneWallMarker");
            RedSealMarker.LoadContent(this.Content, "SummonerMarkers/RedSealMarker");
            SacrificeMarker.LoadContent(this.Content, "SummonerMarkers/SacrificeMarker");
            SkeletonMarker.LoadContent(this.Content, "SummonerMarkers/SkeletonMarker");
            SoulCrushMarker.LoadContent(this.Content, "SummonerMarkers/SoulCrushMarker");
            VampireMarker.LoadContent(this.Content, "SummonerMarkers/VampireMarker");
            VampireBiteMarker.LoadContent(this.Content,
"SummonerMarkers/VampireBiteMarker");
            ZombieMarker.LoadContent(this.Content, "SummonerMarkers/ZombieMarker");

            AssaultMarker.LoadContent(this.Content, "WarriorMarkers/AssaultMarker");
            BackStabMarker.LoadContent(this.Content, "WarriorMarkers/BackStabMarker");
            BleedingMarker.LoadContent(this.Content, "WarriorMarkers/BleedingMarker");
            CrushMarker.LoadContent(this.Content, "WarriorMarkers/CrushMarker");
            DualStrikeMarker.LoadContent(this.Content,
"WarriorMarkers/DualStrikeMarker");
            RapidStabMarker.LoadContent(this.Content, "WarriorMarkers/RapidStabMarker");
            ArrowStormMarker.LoadContent(this.Content,
"WarriorMarkers/ArrowStormMarker");
            SlashMarker.LoadContent(this.Content, "WarriorMarkers/SlashMarker");
            BattleAxeMarker.LoadContent(this.Content, "WarriorMarkers/BattleAxeMarker");
            BerzerkMarker.LoadContent(this.Content, "WarriorMarkers/BerzerkMarker");
            BlockMarker.LoadContent(this.Content, "WarriorMarkers/BlockMarker");
            BloodlustMarker.LoadContent(this.Content, "WarriorMarkers/BloodlustMarker");
            FosseMarker.LoadContent(this.Content, "WarriorMarkers/FosseMarker");
            HealingPotionMarker.LoadContent(this.Content,
"WarriorMarkers/HealingPotionMarker");
            HeroicRemedyMarker.LoadContent(this.Content,
"WarriorMarkers/HeroicRemedyMarker");
            KiteShieldMarker.LoadContent(this.Content,
"WarriorMarkers/KiteShieldMarker");
            LongSwordMarker.LoadContent(this.Content, "WarriorMarkers/LongSwordMarker");
```

```
            MarksmanMarker.LoadContent(this.Content, "WarriorMarkers/MarksmanMarker");
            MournerMarker.LoadContent(this.Content, "WarriorMarkers/MournerMarker");
            PlateMailMarker.LoadContent(this.Content, "WarriorMarkers/PlateMailMarker");
            RaidMarker.LoadContent(this.Content, "WarriorMarkers/RaidMarker");
            RejuvinateMarker.LoadContent(this.Content,
"WarriorMarkers/RejuvinateMarker");
            RemedyMarker.LoadContent(this.Content, "WarriorMarkers/RemedyMarker");
            TowerShieldMarker.LoadContent(this.Content,
"WarriorMarkers/TowerShieldMarker");

            //Created as a spritefont
            Font1 = Content.Load<SpriteFont>("Myfont");
            Font2 = Content.Load<SpriteFont>("Myfont2");

            _currentMouseState = Mouse.GetState();
            _previousMouseState = _currentMouseState;

            //Loading Music
            BackgroundMusic = Content.Load<Song>("Music");
            MediaPlayer.IsRepeating = true;
        }
        #endregion

        //Called when game exits
        #region Unload Content
        protected override void UnloadContent()
        {
            // TODO: Unload any non ContentManager content here
            //This is called whenever the game exits, perhaps
            //saving the score to a nicely formatted output file would be good
        }
        #endregion

        /// <summary> Update Summary
        /// Allows the game to run logic such as updating the world,
        /// checking for collisions, gathering input, and playing audio.
        /// </summary>
        #region Update
        protected override void Update(GameTime gameTime)
        {
            _previousMouseState = _currentMouseState;
            _currentMouseState = Mouse.GetState();
            var mousePosition = new Point(_currentMouseState.X, _currentMouseState.Y);

            #region Exit and Background Music
            // Allows the game to exit
            if (GamePad.GetState(PlayerIndex.One).Buttons.Back == ButtonState.Pressed)
                this.Exit();

            //Allow Background Music to play:
            if (!songstart)
            {
                MediaPlayer.Play(BackgroundMusic);
                songstart = true;
            }
            #endregion

            //Using an enum: GameState, we can easily swap between screens at any time
```

```
switch (state)
{
    #region Game Startup Protocol
    case GameState._StartMenu:
        {
            /* This particular screen displays the various decks
             * and allows the user to pick a deck of their choice */

            //check if mouse is inside the Warrior rectangle
            if (warArea.Contains(mousePosition))
            {
                //To communicate with Draw function
                WarHover = true;

                //check for button press, then do logic
                if (_previousMouseState.LeftButton == ButtonState.Released
                    && _currentMouseState.LeftButton == ButtonState.Pressed)
                {
                    MakeWarriorDeck(Human);
                    WarHover = false;
                    state = GameState._AILevelPick;
                    break;
                }
            }
            else WarHover = false;

            if (sorArea.Contains(mousePosition))
            {
                SorcHover = true;

                if (_previousMouseState.LeftButton == ButtonState.Released
                    && _currentMouseState.LeftButton == ButtonState.Pressed)
                {
                    MakeSorcererDeck(Human);
                    SorcHover = false;
                    state = GameState._AILevelPick;
                    break;
                }
            }
            else SorcHover = false;

            if (sumArea.Contains(mousePosition))
            {
                SumHover = true;

                if (_previousMouseState.LeftButton == ButtonState.Released
                    && _currentMouseState.LeftButton == ButtonState.Pressed)
                {
                    MakeSummonerDeck(Human);
                    SumHover = false;
                    state = GameState._AILevelPick;
                    break;
                }
            }
            else SumHover = false;
            break;
        }
    #endregion
```

```
#region AI Level Pick Screen
case GameState._AILevelPick:
    {
        if (L1Area.Contains(mousePosition))
        {
            AI1Hover = true;
            if (_previousMouseState.LeftButton == ButtonState.Released
                && _currentMouseState.LeftButton == ButtonState.Pressed)
            {
                AI.setLevel(1);
                //Now we need to fill the AI's deck,
                //same way as we did the Human
                MakeWarriorDeck(AI);
                AI1Hover = false;
                state = GameState._GameStart;
                break;
            }
        }
        else AI1Hover = false;

        if (L2Area.Contains(mousePosition))
        {
            AI2Hover = true;
            if (_previousMouseState.LeftButton == ButtonState.Released
                && _currentMouseState.LeftButton == ButtonState.Pressed)
            {
                AI.setLevel(2);
                MakeSorcererDeck(AI);
                AI2Hover = false;
                state = GameState._GameStart;
                break;
            }
        }
        else AI2Hover = false;

        if (L3Area.Contains(mousePosition))
        {
            AI3Hover = true;
            if (_previousMouseState.LeftButton == ButtonState.Released
                && _currentMouseState.LeftButton == ButtonState.Pressed)
            {
                AI.setLevel(3);
                MakeSummonerDeck(AI);
                AI3Hover = false;
                state = GameState._GameStart;
                break;
            }
        }
        else AI3Hover = false;
        break;
    }
#endregion

#region After GameStart
case GameState._GameStart:
    {
        //Always check to see if we have a winner first:
```

```
                        if (Human.getHealthAsInt() <= 0 || AI.getHealthAsInt() <= 0)
                        {
                            state = GameState._GameEnd;
                            break;
                        }

                        //Want to always also check if health is > 40,
                        //because we do not want it to exceed maximum.
                        if (Human.getHealthAsInt() > 40)
                        {
                            Human.setHealth(40);
                        }
                        if (AI.getHealthAsInt() > 40)
                        {
                            AI.setHealth(40);
                        }

                        if (DeckRec.Contains(mousePosition))
                        {
                            DescriptionText = "This is your deck, click to end turn \n
after playing a card.";
                            if (HumanTurn == false) //This is if it is the AI's turn
                            {
                                if (_previousMouseState.LeftButton ==
ButtonState.Released
                                    && _currentMouseState.LeftButton ==
ButtonState.Pressed)
                                {
                                    //If we click the deck once the Humans Turn is over,
                                    //we want to begin AI PlayCard functions, then make
it
                                    //player's turn again
                                    PlayerPlayCard(AI.AITurn(CurrentCard[0],
Human.getHealthAsInt(), Human.getMarkerCount()), AI, Human);
                                    HumanTurn = true;
                                }
                            }
                        }
                        else if (ExitRec.Contains(mousePosition))
                        {
                            DescriptionText = "If you want to exit the game/forfeit \n
click here. This will exit the game.";
                            if (_previousMouseState.LeftButton == ButtonState.Released
                                && _currentMouseState.LeftButton == ButtonState.Pressed)
                            {
                                this.Exit();
                            }
                        }
                        else if (HM1Area.Contains(mousePosition))
                        {
                            if(Human.getMarkerCount() >= 1)
                                DescriptionText =
Human.getFromMarkers(0).getDescription();
                        }
                        else if (HM2Area.Contains(mousePosition))
                        {
                            if (Human.getMarkerCount() >= 2)
```

```
                                DescriptionText =
Human.getFromMarkers(1).getDescription();
                            }
                            else if (HM3Area.Contains(mousePosition))
                            {
                                if (Human.getMarkerCount() == 3)
                                    DescriptionText =
Human.getFromMarkers(2).getDescription();
                            }
                            else if (AIM1Area.Contains(mousePosition))
                            {
                                if (AI.getMarkerCount() >= 1)
                                    DescriptionText = AI.getFromMarkers(0).getDescription();
                            }
                            else if (AIM2Area.Contains(mousePosition))
                            {
                                if (AI.getMarkerCount() >= 2)
                                    DescriptionText = AI.getFromMarkers(1).getDescription();
                            }
                            else if (AIM3Area.Contains(mousePosition))
                            {
                                if (AI.getMarkerCount() == 3)
                                    DescriptionText = AI.getFromMarkers(2).getDescription();
                            }
                            else if (CurrentCardRec.Contains(mousePosition))
                            {
                                if (CurrentCard.Count() > 0)
                                    DescriptionText = CurrentCard[0].getDescription();
                            }
                            else DescriptionText = DefaultDesText;

                            if (Card1Area.Contains(mousePosition))
                            {
                                Card1Hover = true;
                                if (_previousMouseState.LeftButton == ButtonState.Released
                                    && _currentMouseState.LeftButton == ButtonState.Pressed)
                                {
                                    if (HumanTurn)
                                    {
                                        PlayerPlayCard(Human.getFromHand(0), Human, AI);
                                        HumanTurn = false;
                                    }
                                }
                            }
                            else Card1Hover = false;

                            if (Card2Area.Contains(mousePosition))
                            {
                                Card2Hover = true;
                                if (_previousMouseState.LeftButton == ButtonState.Released
                                    && _currentMouseState.LeftButton == ButtonState.Pressed)
                                {
                                    if (HumanTurn)
                                    {
                                        PlayerPlayCard(Human.getFromHand(1), Human, AI);
                                        HumanTurn = false;
                                    }
                                }
```

```
            }
            else Card2Hover = false;

            if (Card3Area.Contains(mousePosition))
            {
                Card3Hover = true;
                if (_previousMouseState.LeftButton == ButtonState.Released
                    && _currentMouseState.LeftButton == ButtonState.Pressed)
                {
                    if (HumanTurn)
                    {
                        PlayerPlayCard(Human.getFromHand(2), Human, AI);
                        HumanTurn = false;
                    }
                }
            }
            else Card3Hover = false;

            if (Card4Area.Contains(mousePosition))
            {
                Card4Hover = true;
                if (_previousMouseState.LeftButton == ButtonState.Released
                    && _currentMouseState.LeftButton == ButtonState.Pressed)
                {
                    if (HumanTurn)
                    {
                        PlayerPlayCard(Human.getFromHand(3), Human, AI);
                        HumanTurn = false;
                    }
                }
            }
            else Card4Hover = false;

            if (Card5Area.Contains(mousePosition))
            {
                Card5Hover = true;
                if (_previousMouseState.LeftButton == ButtonState.Released
                    && _currentMouseState.LeftButton == ButtonState.Pressed)
                {
                    if (HumanTurn)
                    {
                        PlayerPlayCard(Human.getFromHand(4), Human, AI);
                        HumanTurn = false;
                    }
                }
            }
            else Card5Hover = false;
            break;
        }
    #endregion

    //When the game ends, we simply exit.
    #region Game Over
    case GameState._GameEnd:
        {
            this.Exit();
            break;
        }
```

```
        #endregion
    }

    base.Update(gameTime);
}
#endregion

/// <summary> Draw Summary
/// Draws every object in the game. All of the backgrounds,
/// all of the cards, etc. Update will change booleans to
/// tell Draw exactly what to draw when.
/// </summary>
#region Draw Method
protected override void Draw(GameTime gameTime)
{
    GraphicsDevice.Clear(Color.DarkGray);


    switch (state)
    {
        #region Opening Game Screen
        case GameState._StartMenu:
            {
                //Need to draw the On-Hover items
                startBatch.Begin();
                if (WarHover)
                    WarriorHover.Draw(this.startBatch);
                else if (SorcHover)
                    SorcererHover.Draw(this.startBatch);
                else if (SumHover)
                    SummonerHover.Draw(this.startBatch);
                else
                    StartMenu.Draw(this.startBatch);
                startBatch.End();
                break;
            }
        #endregion

        #region AI Level Screen Drawing
        case GameState._AILevelPick:
            {
                AILevelBatch.Begin();
                if (AI1Hover)
                    AILevel1Screen.Draw(this.AILevelBatch);
                else if (AI2Hover)
                    AILevel2Screen.Draw(this.AILevelBatch);
                else if (AI3Hover)
                    AILevel3Screen.Draw(this.AILevelBatch);
                else
                    AIBaseLevelScreen.Draw(this.AILevelBatch);
                AILevelBatch.End();
                break;
            }
        #endregion

        #region Actual Gameplay Screen
        case GameState._GameStart:
            {
```

```
                        //begin drawing sprites, in order
                        spriteBatch.Begin();
                        background.Draw(this.spriteBatch);
                        #region Draw Cards In Hand
                        //Draws cards with Sprite functions
                        sp1 = CardToDraw(Human.getFromHand(0).getSpriteName());
                        sp1.Position = PHandS1;
                        sp1.Draw(this.spriteBatch);

                        sp2 = CardToDraw(Human.getFromHand(1).getSpriteName());
                        sp2.Position = PHandS2;
                        sp2.Draw(this.spriteBatch);

                        sp3 = CardToDraw(Human.getFromHand(2).getSpriteName());
                        sp3.Position = PHandS3;
                        sp3.Draw(this.spriteBatch);

                        sp4 = CardToDraw(Human.getFromHand(3).getSpriteName());
                        sp4.Position = PHandS4;
                        sp4.Draw(this.spriteBatch);

                        sp5 = CardToDraw(Human.getFromHand(4).getSpriteName());
                        sp5.Position = PHandS5;
                        sp5.Draw(this.spriteBatch);
                        #endregion

                        #region Draw Current Card
                        if (CurrentCard.Count() > 0)
                        {
                            Curr = CardToDraw(CurrentCard[CurrentCard.Count() -
1].getSpriteName());

                            Curr.Position = CCPosition;
                            Curr.Draw(this.spriteBatch);
                        }
                        #endregion

                        #region Draw Markers (Human and AI)
                        //Human
                        if (Human.getMarkerCount() == 1)
                        {
                            HM1 = MarkerToDraw(Human.getFromMarkers(0).getSpriteName());
                            HM1.Position = HM1pos;
                            HM1.Draw(this.spriteBatch);
                        }
                        if (Human.getMarkerCount() == 2)
                        {
                            HM1 = MarkerToDraw(Human.getFromMarkers(0).getSpriteName());
                            HM1.Position = HM1pos;
                            HM1.Draw(this.spriteBatch);
                            HM2 = MarkerToDraw(Human.getFromMarkers(1).getSpriteName());
                            HM2.Position = HM2pos;
                            HM2.Draw(this.spriteBatch);
                        }
                        if (Human.getMarkerCount() == 3)
                        {
                            HM1 = MarkerToDraw(Human.getFromMarkers(0).getSpriteName());
                            HM1.Position = HM1pos;
                            HM1.Draw(this.spriteBatch);
```

```
            HM2 = MarkerToDraw(Human.getFromMarkers(1).getSpriteName());
            HM2.Position = HM2pos;
            HM2.Draw(this.spriteBatch);
            HM3 = MarkerToDraw(Human.getFromMarkers(2).getSpriteName());
            HM3.Position = HM3pos;
            HM3.Draw(this.spriteBatch);
        }

        //AI markers - note this must all be in DRAW
        if (AI.getMarkerCount() == 1)
        {
            AIM1 = MarkerToDraw(AI.getFromMarkers(0).getSpriteName());
            AIM1.Position = AIM1pos;
            AIM1.Draw(this.spriteBatch);
        }
        if (AI.getMarkerCount() == 2)
        {
            AIM1 = MarkerToDraw(AI.getFromMarkers(0).getSpriteName());
            AIM1.Position = AIM1pos;
            AIM1.Draw(this.spriteBatch);
            AIM2 = MarkerToDraw(AI.getFromMarkers(1).getSpriteName());
            AIM2.Position = AIM2pos;
            AIM2.Draw(this.spriteBatch);
        }
        if (AI.getMarkerCount() == 3)
        {
            AIM1 = MarkerToDraw(AI.getFromMarkers(0).getSpriteName());
            AIM1.Position = AIM1pos;
            AIM1.Draw(this.spriteBatch);
            AIM2 = MarkerToDraw(AI.getFromMarkers(1).getSpriteName());
            AIM2.Position = AIM2pos;
            AIM2.Draw(this.spriteBatch);
            AIM3 = MarkerToDraw(AI.getFromMarkers(2).getSpriteName());
            AIM3.Position = AIM3pos;
            AIM3.Draw(this.spriteBatch);
        }
        #endregion

        #region Card Hovers
        //BIG IF() because it IS ABSOLUTELY faster than NAME comparison
using variable names.
        //Also this code WILL NOT break in later versions of XNA.
        if (Card1Hover)
            DescriptionText = Human.getFromHand(0).getDescription();
        if (Card2Hover)
            DescriptionText = Human.getFromHand(1).getDescription();
        if (Card3Hover)
            DescriptionText = Human.getFromHand(2).getDescription();
        if (Card4Hover)
            DescriptionText = Human.getFromHand(3).getDescription();
        if (Card5Hover)
            DescriptionText = Human.getFromHand(4).getDescription();
        #endregion

        #region Description, and any other Textbox
        //Draw this last
```

```
                        //Find the center of the Description String to then draw it
neatly
                        Vector2 FontOrigin = Font1.MeasureString(DescriptionText) / 2;
                        spriteBatch.DrawString(Font1, DescriptionText, DescriptionPos,
Color.White,
            0, FontOrigin, 1.0f, SpriteEffects.None, 0.5f); //DescriptionText is what
gets changed. Position is fixed.

                        //draw the HP values for AI and Player.
                        //NOTE: These are drawn each iteration, so when changed, Draw
updates automatically.
                        Vector2 PHOrigin = Font2.MeasureString(Human.getHealth()) / 2;
                        spriteBatch.DrawString(Font2, Human.getHealth(), PHealthVec,
Color.Yellow,
            0, PHOrigin, 1.3f, SpriteEffects.None, 0.5f);
                        Vector2 AIHOrigin = Font2.MeasureString(AI.getHealth()) / 2;
                        spriteBatch.DrawString(Font2, AI.getHealth(), AIHealthVec,
Color.Yellow,
            0, AIHOrigin, 1.3f, SpriteEffects.None, 0.5f);
                        #endregion
                        spriteBatch.End();
                        break;
                    }
                #endregion

                #region Game End Screen
                case GameState._GameEnd:
                    {
                        //TODO: Drawing for end game, or perhaps restart
                        break;
                    }
                #endregion
            }

            base.Draw(gameTime);
        }
        #endregion

        #region String to Sprite Functions (CardToDraw/MarkerToDraw)
        //This is necessary so that we can convery a string to a Sprite, as each card
        //can not have a "Sprite" attribute, we had to have a string that converts to
sprite
        Sprite CardToDraw(string aString)
        {
            if (aString == "AirAura")
                return AirAura;
            else if (aString == "BladeStorm")
                return BladeStorm;
            else if (aString == "Bless")
                return Bless;
            else if (aString == "Blizzard")
                return Blizzard;
            else if (aString == "ChainLightning")
                return ChainLightning;
            else if (aString == "DeathNode")
                return DeathNode;
            else if (aString == "EarthAura")
                return EarthAura;
```

```
else if (aString == "Earthquake")
    return Earthquake;
else if (aString == "FireAura")
    return FireAura;
else if (aString == "Firewall")
    return Firewall;
else if (aString == "HealingScroll")
    return HealingScroll;
else if (aString == "HealingShower")
    return HealingShower;
else if (aString == "IceBolt")
    return IceBolt;
else if (aString == "LifeNode")
    return LifeNode;
else if (aString == "MoonNode")
    return MoonNode;
else if (aString == "RageofNature")
    return RageofNature;
else if (aString == "Sandstorm")
    return Sandstorm;
else if (aString == "Starfall")
    return Starfall;
else if (aString == "SunNode")
    return SunNode;
else if (aString == "WaterAura")
    return WaterAura;
else if (aString == "Whirlwind")
    return Whirlwind;
else if (aString == "Wildfire")
    return Wildfire;
else if (aString == "Willowisp")
    return Willowisp;
else if (aString == "AcidRain")
    return AcidRain;
else if (aString == "Atrophy")
    return Atrophy;
else if (aString == "Blackout")
    return Blackout;
else if (aString == "Disease")
    return Disease;
else if (aString == "Golem")
    return Golem;
else if (aString == "LifeBlood")
    return LifeBlood;
else if (aString == "SoulPower")
    return SoulPower;
else if (aString == "Bats")
    return Bats;
else if (aString == "BoneWall")
    return BoneWall;
else if (aString == "DarkTouch")
    return DarkTouch;
else if (aString == "Fang")
    return Fang;
else if (aString == "Ghoul")
    return Ghoul;
else if (aString == "Graveyard")
    return Graveyard;
```

```
else if (aString == "Parasite")
    return Parasite;
else if (aString == "PoisonCloud")
    return PoisonCloud;
else if (aString == "RedBoneWall")
    return RedBoneWall;
else if (aString == "RedSeal")
    return RedSeal;
else if (aString == "Sacrifice")
    return Sacrifice;
else if (aString == "Skeleton")
    return Skeleton;
else if (aString == "SoulCrush")
    return SoulCrush;
else if (aString == "Vampire")
    return Vampire;
else if (aString == "VampireBite")
    return VampireBite;
else if (aString == "Zombie")
    return Zombie;
else if (aString == "Assault")
    return Assault;
else if (aString == "ArrowStorm")
    return ArrowStorm;
else if (aString == "BackStab")
    return BackStab;
else if (aString == "BattleAxe")
    return BattleAxe;
else if (aString == "Berzerk")
    return Berzerk;
else if (aString == "Bleeding")
    return Bleeding;
else if (aString == "Block")
    return Block;
else if (aString == "Bloodlust")
    return Bloodlust;
else if (aString == "Crush")
    return Crush;
else if (aString == "RapidStab")
    return RapidStab;
else if (aString == "DualStrike")
    return DualStrike;
else if (aString == "Fosse")
    return Fosse;
else if (aString == "HealingPotion")
    return HealingPotion;
else if (aString == "HeroicRemedy")
    return HeroicRemedy;
else if (aString == "KiteShield")
    return KiteShield;
else if (aString == "LongSword")
    return LongSword;
else if (aString == "Marksman")
    return Marksman;
else if (aString == "Mourner")
    return Mourner;
else if (aString == "PlateMail")
    return PlateMail;
```

```
    else if (aString == "Raid")
        return Raid;
    else if (aString == "Rejuvinate")
        return Rejuvinate;
    else if (aString == "Remedy")
        return Remedy;
    else if (aString == "Slash")
        return Slash;
    else if (aString == "TowerShield")
        return TowerShield;
    else
        return DefaultCard; //This will display if a card was spelled wrong
}

Sprite MarkerToDraw(string aString)
{
    if (aString == "AirAura")
        return AirAuraMarker;
    else if (aString == "BladeStorm")
        return BladeStormMarker;
    else if (aString == "Bless")
        return BlessMarker;
    else if (aString == "Blizzard")
        return BlizzardMarker;
    else if (aString == "ChainLightning")
        return ChainLightningMarker;
    else if (aString == "DeathNode")
        return DeathNodeMarker;
    else if (aString == "EarthAura")
        return EarthAuraMarker;
    else if (aString == "Earthquake")
        return EarthquakeMarker;
    else if (aString == "FireAura")
        return FireAuraMarker;
    else if (aString == "Firewall")
        return FirewallMarker;
    else if (aString == "HealingScroll")
        return HealingScrollMarker;
    else if (aString == "HealingShower")
        return HealingShowerMarker;
    else if (aString == "IceBolt")
        return IceBoltMarker;
    else if (aString == "LifeNode")
        return LifeNodeMarker;
    else if (aString == "MoonNode")
        return MoonNodeMarker;
    else if (aString == "RageofNature")
        return RageofNatureMarker;
    else if (aString == "Sandstorm")
        return SandstormMarker;
    else if (aString == "Starfall")
        return StarfallMarker;
    else if (aString == "SunNode")
        return SunNodeMarker;
    else if (aString == "WaterAura")
        return WaterAuraMarker;
    else if (aString == "Whirlwind")
        return WhirlwindMarker;
```

```
else if (aString == "Wildfire")
    return WildfireMarker;
else if (aString == "Willowisp")
    return WillowispMarker;
else if (aString == "AcidRain")
    return AcidRainMarker;
else if (aString == "Atrophy")
    return AtrophyMarker;
else if (aString == "Blackout")
    return BlackoutMarker;
else if (aString == "Disease")
    return DiseaseMarker;
else if (aString == "Golem")
    return GolemMarker;
else if (aString == "LifeBlood")
    return LifeBloodMarker;
else if (aString == "SoulPower")
    return SoulPowerMarker;
else if (aString == "Bats")
    return BatsMarker;
else if (aString == "BoneWall")
    return BoneWallMarker;
else if (aString == "DarkTouch")
    return DarkTouchMarker;
else if (aString == "Fang")
    return FangMarker;
else if (aString == "Ghoul")
    return GhoulMarker;
else if (aString == "Graveyard")
    return GraveyardMarker;
else if (aString == "Parasite")
    return ParasiteMarker;
else if (aString == "PoisonCloud")
    return PoisonCloudMarker;
else if (aString == "RedBoneWall")
    return RedBoneWallMarker;
else if (aString == "RedSeal")
    return RedSealMarker;
else if (aString == "Sacrifice")
    return SacrificeMarker;
else if (aString == "Skeleton")
    return SkeletonMarker;
else if (aString == "SoulCrush")
    return SoulCrushMarker;
else if (aString == "Vampire")
    return VampireMarker;
else if (aString == "VampireBite")
    return VampireBiteMarker;
else if (aString == "Zombie")
    return ZombieMarker;
else if (aString == "Assault")
    return AssaultMarker;
else if (aString == "ArrowStorm")
    return ArrowStormMarker;
else if (aString == "BackStab")
    return BackStabMarker;
else if (aString == "BattleAxe")
    return BattleAxeMarker;
```

```
    else if (aString == "Berzerk")
        return BerzerkMarker;
    else if (aString == "Bleeding")
        return BleedingMarker;
    else if (aString == "Block")
        return BlockMarker;
    else if (aString == "Bloodlust")
        return BloodlustMarker;
    else if (aString == "Crush")
        return CrushMarker;
    else if (aString == "RapidStab")
        return RapidStabMarker;
    else if (aString == "DualStrike")
        return DualStrikeMarker;
    else if (aString == "Fosse")
        return FosseMarker;
    else if (aString == "HealingPotion")
        return HealingPotionMarker;
    else if (aString == "HeroicRemedy")
        return HeroicRemedyMarker;
    else if (aString == "KiteShield")
        return KiteShieldMarker;
    else if (aString == "LongSword")
        return LongSwordMarker;
    else if (aString == "Marksman")
        return MarksmanMarker;
    else if (aString == "Mourner")
        return MournerMarker;
    else if (aString == "PlateMail")
        return PlateMailMarker;
    else if (aString == "Raid")
        return RaidMarker;
    else if (aString == "Rejuvinate")
        return RejuvinateMarker;
    else if (aString == "Remedy")
        return RemedyMarker;
    else if (aString == "Slash")
        return SlashMarker;
    else if (aString == "TowerShield")
        return TowerShieldMarker;
    else
        return null;
}
#endregion

#region Sees If Minion, reduces reused code
//This function sees if the player has a minion and does damage properly
//it takes a count, damage, and player (by ref) and it functions accordingly.
int PlayCardHelper(int count, int damage, Player aPlayer){
    if (count == 3)
    {
        if (aPlayer.getFromMarkers(2).SeeIfMinion() == true)
        {
            if (aPlayer.getFromMarkers(2).getMinionHealth() > damage)
            {
                aPlayer.getFromMarkers(2).LoseMinionHealth(damage);
                return 0;
            }
```

```
        else
        {
            damage = damage - aPlayer.getFromMarkers(2).getMinionHealth();
            aPlayer.RemoveMarkerAt(2);
        }
    }
    if (aPlayer.getFromMarkers(1).SeeIfMinion() == true)
    {
        if (aPlayer.getFromMarkers(1).getMinionHealth() > damage)
        {
            aPlayer.getFromMarkers(1).LoseMinionHealth(damage);
            return 0;
        }
        else
        {
            damage = damage - aPlayer.getFromMarkers(1).getMinionHealth();
            aPlayer.RemoveMarkerAt(1);
        }
    }
    if (aPlayer.getFromMarkers(0).SeeIfMinion() == true)
    {
        if (aPlayer.getFromMarkers(0).getMinionHealth() > damage)
        {
            aPlayer.getFromMarkers(0).LoseMinionHealth(damage);
            return 0;
        }
        else
        {
            damage = damage - aPlayer.getFromMarkers(0).getMinionHealth();
            aPlayer.RemoveMarkerAt(0);
        }
    }
    return damage;
}

if (count == 2)
{
    if (aPlayer.getFromMarkers(1).SeeIfMinion() == true)
    {
        if (aPlayer.getFromMarkers(1).getMinionHealth() > damage)
        {
            aPlayer.getFromMarkers(1).LoseMinionHealth(damage);
            return 0;
        }
        else
        {
            damage = damage - aPlayer.getFromMarkers(1).getMinionHealth();
            aPlayer.RemoveMarkerAt(1);
        }
    }
    if (aPlayer.getFromMarkers(0).SeeIfMinion() == true)
    {
        if (aPlayer.getFromMarkers(0).getMinionHealth() > damage)
        {
            aPlayer.getFromMarkers(0).LoseMinionHealth(damage);
            return 0;
        }
        else
```

```
            {
                damage = damage - aPlayer.getFromMarkers(0).getMinionHealth();
                aPlayer.RemoveMarkerAt(0);
            }
        }
        return damage;
    }

    if (count == 1)
    {
        if (aPlayer.getFromMarkers(0).SeeIfMinion() == true)
        {
            if (aPlayer.getFromMarkers(0).getMinionHealth() > damage)
            {
                aPlayer.getFromMarkers(0).LoseMinionHealth(damage);
                return 0;
            }
            else
            {
                damage = damage - aPlayer.getFromMarkers(0).getMinionHealth();
                aPlayer.RemoveMarkerAt(0);
            }
        }
        return damage;
    }
    //otherwise if they don't have minions:
    return damage;
}
#endregion

//There are a Human and AI that both use this
#region Player PlayCard
//C# treats objects like pointers, so we can send byRef automatically
//aPlayer = the player playing a card
//bPlayer = the player defending
void PlayerPlayCard(Card aCard, Player aPlayer, Player bPlayer)
{
    #region Deal with Current Card
    //Put currentcard in deck if there is one, then clear it and replace
    //Note, we put in "bPlayer" deck, because that was the last player
    //to play a card into current
    if (CurrentCard.Count() == 1)
    {
        bPlayer.AddToDeck(CurrentCard[0]);
        CurrentCard.Clear();
    }
    CurrentCard.Add(aCard);
    #endregion

    #region If Minion or Temporary
    if (aCard.SeeIfMinion() || aCard.SeeIfTemp())
    {
        if (aPlayer.getMarkerCount() == 3)
        {
            aPlayer.RemoveMarker();
        }
        aPlayer.AddToMarkers(aCard);
        aPlayer.RemoveHand(aCard);
```

```
                aPlayer.DrawACard();
            }
            #endregion

            //Deal with markers now
            #region Markers
            if (aPlayer.getMarkerCount() == 1)
            {
                aPlayer.GainHealth(aPlayer.getFromMarkers(0).getHeal()); //first get
healed by marker
                int damage = aPlayer.getFromMarkers(0).getDamageThem();
                if (damage > 0)
                {
                    damage = PlayCardHelper(bPlayer.getMarkerCount(), damage, bPlayer);
                    bPlayer.LoseHealth(damage);
                    //Cards with 0 turns are not affected by turn loss
                    if (aPlayer.getFromMarkers(0).UsesTurns())
                    {
                        //if there are turns left, reduce the turns for marker
                        aPlayer.getFromMarkers(0).LoseTurn();
                        //then check if 0 (delete if true)
                        if (aPlayer.getFromMarkers(0).getTurns() == 0)
                            aPlayer.RemoveMarkerAt(0);
                    }
                }
            }
            if (aPlayer.getMarkerCount() == 2)
            {
                aPlayer.GainHealth(aPlayer.getFromMarkers(1).getHeal());
                int damage2 = aPlayer.getFromMarkers(1).getDamageThem();
                if (damage2 > 0)
                {
                    damage2 = PlayCardHelper(bPlayer.getMarkerCount(), damage2, bPlayer);
                    bPlayer.LoseHealth(damage2);
                    if (aPlayer.getFromMarkers(1).UsesTurns())
                    {
                        aPlayer.getFromMarkers(1).LoseTurn();
                        if (aPlayer.getFromMarkers(1).getTurns() == 0)
                            aPlayer.RemoveMarkerAt(1);
                    }
                }
                aPlayer.GainHealth(aPlayer.getFromMarkers(0).getHeal());
                int damage = aPlayer.getFromMarkers(0).getDamageThem();
                if (damage > 0)
                {
                    damage = PlayCardHelper(bPlayer.getMarkerCount(), damage, bPlayer);
                    bPlayer.LoseHealth(damage);
                    if (aPlayer.getFromMarkers(0).UsesTurns())
                    {
                        aPlayer.getFromMarkers(0).LoseTurn();
                        if (aPlayer.getFromMarkers(0).getTurns() == 0)
                            aPlayer.RemoveMarkerAt(0);
                    }
                }
            }
            if (aPlayer.getMarkerCount() == 3)
            {
                aPlayer.GainHealth(aPlayer.getFromMarkers(2).getHeal());
```

```
            int damage3 = aPlayer.getFromMarkers(2).getDamageThem();
            if (damage3 > 0)
            {
                damage3 = PlayCardHelper(bPlayer.getMarkerCount(), damage3, bPlayer);
                bPlayer.LoseHealth(damage3);
                if (aPlayer.getFromMarkers(2).UsesTurns())
                {
                    aPlayer.getFromMarkers(2).LoseTurn();
                    if (aPlayer.getFromMarkers(2).getTurns() == 0)
                        aPlayer.RemoveMarkerAt(2);
                }
            }
            aPlayer.GainHealth(aPlayer.getFromMarkers(1).getHeal());
            int damage2 = aPlayer.getFromMarkers(1).getDamageThem();
            if (damage2 > 0)
            {
                damage2 = PlayCardHelper(bPlayer.getMarkerCount(), damage2, bPlayer);
                bPlayer.LoseHealth(damage2);
                if (aPlayer.getFromMarkers(1).UsesTurns())
                {
                    aPlayer.getFromMarkers(1).LoseTurn();
                    if (aPlayer.getFromMarkers(1).getTurns() == 0)
                        aPlayer.RemoveMarkerAt(1);
                }
            }
            aPlayer.GainHealth(aPlayer.getFromMarkers(0).getHeal());
            int damage = aPlayer.getFromMarkers(0).getDamageThem();
            if (damage > 0)
            {
                damage = PlayCardHelper(bPlayer.getMarkerCount(), damage, bPlayer);
                bPlayer.LoseHealth(damage);
                if (aPlayer.getFromMarkers(0).UsesTurns())
                {
                    aPlayer.getFromMarkers(0).LoseTurn();
                    if (aPlayer.getFromMarkers(0).getTurns() == 0)
                        aPlayer.RemoveMarkerAt(0);
                }
            }
            if (aCard.SeeIfTemp() || aCard.SeeIfMinion()) return;
        }
        #endregion

        #region Is it a Normal card?
        if (aCard.SeeIfNormal())
        {
            //Either lose health, gain health, deal damage, or any combination
            aPlayer.GainHealth(aCard.getHeal());
            aPlayer.LoseHealth(aCard.getDamageMe());
            int healthloss = PlayCardHelper(bPlayer.getMarkerCount(),
aCard.getDamageThem(),
                bPlayer);
            bPlayer.LoseHealth(healthloss);
            //still remove from hand and draw
            aPlayer.RemoveHand(aCard);
            aPlayer.DrawACard();
            return;
        }
        #endregion
```

```
    #region Does it ignore Markers?
    if (aCard.DoesDisableMarker())
    {
        //then all we want to do is direct damage, ignore all defenses
        bPlayer.LoseHealth(aCard.getDamageThem());
        aPlayer.RemoveHand(aCard);
        aPlayer.DrawACard();
        return;
    }
    #endregion

    #region Does it Remove Marker?
    if (aCard.DoesRemoveMarker())
    {
        //another special type of card, removes markers
        if (aCard.getMarkersRemoved() == 1)
        {
            if (bPlayer.getMarkerCount() >= 1)
            {
                bPlayer.RemoveMarker();
            }
        }
        else //it will remove all markers in this case
        {
            while (AI.getMarkerCount() > 0)
            {
                bPlayer.RemoveMarker();
            }
        }
        aPlayer.RemoveHand(aCard);
        aPlayer.DrawACard();
        return;
    }
    #endregion
}
#endregion

//Reusable code for creating a deck
#region Warrior Deck Creation
void MakeWarriorDeck(Player aPlayer)
{
    if (aPlayer.DeckEmpty()) //so long as the deck is empty
    {
        //Card objects
        Card w1 = new Card();
        w1.setAll(1, 3);
        aPlayer.AddToDeck(w1);
        aPlayer.AddToDeck(w1);
        aPlayer.AddToDeck(w1);

        Card w2 = new Card();
        w2.setAll(2, 3);
        aPlayer.AddToDeck(w2);
        aPlayer.AddToDeck(w2);
        aPlayer.AddToDeck(w2);

        Card w3 = new Card();
```

```
w3.setAll(3, 3);
aPlayer.AddToDeck(w3);
aPlayer.AddToDeck(w3);
aPlayer.AddToDeck(w3);

Card w4 = new Card();
w4.setAll(4, 3);
aPlayer.AddToDeck(w4);
aPlayer.AddToDeck(w4);
aPlayer.AddToDeck(w4);

Card w5 = new Card();
w5.setAll(5, 3);
aPlayer.AddToDeck(w5);
aPlayer.AddToDeck(w5);
aPlayer.AddToDeck(w5);

Card w6 = new Card();
w6.setAll(6, 3);
aPlayer.AddToDeck(w6);
aPlayer.AddToDeck(w6);

Card w7 = new Card();
w7.setAll(7, 3);
aPlayer.AddToDeck(w7);
aPlayer.AddToDeck(w7);
aPlayer.AddToDeck(w7);

Card w8 = new Card();
w8.setAll(8, 3);
aPlayer.AddToDeck(w8);

Card w9 = new Card();
w9.setAll(9, 3);
aPlayer.AddToDeck(w9);
aPlayer.AddToDeck(w9);

Card w10 = new Card();
w10.setAll(10, 3);
aPlayer.AddToDeck(w10);
aPlayer.AddToDeck(w10);
aPlayer.AddToDeck(w10);

Card w11 = new Card();
w11.setAll(11, 3);
aPlayer.AddToDeck(w11);
aPlayer.AddToDeck(w11);
aPlayer.AddToDeck(w11);

Card w12 = new Card();
w12.setAll(12, 3);
aPlayer.AddToDeck(w12);
aPlayer.AddToDeck(w12);

Card w13 = new Card();
w13.setAll(13, 3);
aPlayer.AddToDeck(w13);
```

```
        Card w14 = new Card();
        w14.setAll(14, 3);
        aPlayer.AddToDeck(w14);

        Card w15 = new Card();
        w15.setAll(15, 3);
        aPlayer.AddToDeck(w15);
        aPlayer.AddToDeck(w15);
        aPlayer.AddToDeck(w15);

        Card w16 = new Card();
        w16.setAll(16, 3);
        aPlayer.AddToDeck(w16);
        aPlayer.AddToDeck(w16);
        aPlayer.AddToDeck(w16);

        Card w17 = new Card();
        w17.setAll(17, 3);
        aPlayer.AddToDeck(w17);
        aPlayer.AddToDeck(w17);

        Card w18 = new Card();
        w18.setAll(18, 3);
        aPlayer.AddToDeck(w18);

        Card w19 = new Card();
        w19.setAll(19, 3);
        aPlayer.AddToDeck(w19);
        aPlayer.AddToDeck(w19);
        aPlayer.AddToDeck(w19);

        Card w20 = new Card();
        w20.setAll(20, 3);
        aPlayer.AddToDeck(w20);
        aPlayer.AddToDeck(w20);
        aPlayer.AddToDeck(w20);

        Card w21 = new Card();
        w21.setAll(21, 3);
        aPlayer.AddToDeck(w21);

        Card w22 = new Card();
        w22.setAll(22, 3);
        aPlayer.AddToDeck(w22);

        //shuffles the deck based on shuffle algorithm
        aPlayer.Shuffle();

        /* Puts first 5 cards from deck into hand,
         * automatically removes from deck.*/
        for (int i = 0; i < 5; i++) aPlayer.DrawACard();
    }
}
#endregion

#region Sorcerer Deck Creation
void MakeSorcererDeck(Player aPlayer)
{
```

```
if (aPlayer.DeckEmpty())
{
    Card s1 = new Card();
    s1.setAll(1, 2);
    aPlayer.AddToDeck(s1);
    aPlayer.AddToDeck(s1);
    aPlayer.AddToDeck(s1);

    Card s2 = new Card();
    s2.setAll(2, 2);
    aPlayer.AddToDeck(s2);

    Card s3 = new Card();
    s3.setAll(3, 2);
    aPlayer.AddToDeck(s3);
    aPlayer.AddToDeck(s3);
    aPlayer.AddToDeck(s3);

    Card s4 = new Card();
    s4.setAll(4, 2);
    aPlayer.AddToDeck(s4);
    aPlayer.AddToDeck(s4);
    aPlayer.AddToDeck(s4);

    Card s5 = new Card();
    s5.setAll(5, 2);
    aPlayer.AddToDeck(s5);
    aPlayer.AddToDeck(s5);
    aPlayer.AddToDeck(s5);

    Card s6 = new Card();
    s6.setAll(6, 2);
    aPlayer.AddToDeck(s6);

    Card s7 = new Card();
    s7.setAll(7, 2);
    aPlayer.AddToDeck(s7);
    aPlayer.AddToDeck(s7);
    aPlayer.AddToDeck(s7);

    Card s8 = new Card();
    s8.setAll(8, 2);
    aPlayer.AddToDeck(s8);
    aPlayer.AddToDeck(s8);
    aPlayer.AddToDeck(s8);

    Card s9 = new Card();
    s9.setAll(9, 2);
    aPlayer.AddToDeck(s9);
    aPlayer.AddToDeck(s9);
    aPlayer.AddToDeck(s9);

    Card s10 = new Card();
    s10.setAll(10, 2);
    aPlayer.AddToDeck(s10);
    aPlayer.AddToDeck(s10);
    aPlayer.AddToDeck(s10);
```

```
Card s11 = new Card();
s11.setAll(11, 2);
aPlayer.AddToDeck(s11);

Card s12 = new Card();
s12.setAll(12, 2);
aPlayer.AddToDeck(s12);
aPlayer.AddToDeck(s12);
aPlayer.AddToDeck(s12);

Card s13 = new Card();
s13.setAll(13, 2);
aPlayer.AddToDeck(s13);
aPlayer.AddToDeck(s13);
aPlayer.AddToDeck(s13);

Card s14 = new Card();
s14.setAll(14, 2);
aPlayer.AddToDeck(s14);
aPlayer.AddToDeck(s14);
aPlayer.AddToDeck(s14);

Card s15 = new Card();
s15.setAll(15, 2);
aPlayer.AddToDeck(s15);
aPlayer.AddToDeck(s15);

Card s16 = new Card();
s16.setAll(16, 2);
aPlayer.AddToDeck(s16);

Card s17 = new Card();
s17.setAll(17, 2);
aPlayer.AddToDeck(s17);

Card s18 = new Card();
s18.setAll(18, 2);
aPlayer.AddToDeck(s18);
aPlayer.AddToDeck(s18);
aPlayer.AddToDeck(s18);

Card s19 = new Card();
s19.setAll(19, 2);
aPlayer.AddToDeck(s19);
aPlayer.AddToDeck(s19);

Card s20 = new Card();
s20.setAll(20, 2);
aPlayer.AddToDeck(s20);

Card s21 = new Card();
s21.setAll(21, 2);
aPlayer.AddToDeck(s21);

Card s22 = new Card();
s22.setAll(22, 2);
aPlayer.AddToDeck(s22);
aPlayer.AddToDeck(s22);
```

```
            aPlayer.AddToDeck(s22);

            Card s23 = new Card();
            s23.setAll(23, 2);
            aPlayer.AddToDeck(s23);
            aPlayer.AddToDeck(s23);

            aPlayer.Shuffle();

            for (int i = 0; i < 5; i++) aPlayer.DrawACard();
        }
    }
    #endregion

    #region Summoner Deck Creation
    void MakeSummonerDeck(Player aPlayer)
    {
        if (aPlayer.DeckEmpty())
        {
            Card su1 = new Card();
            su1.setAll(1, 1);
            aPlayer.AddToDeck(su1);
            aPlayer.AddToDeck(su1);
            aPlayer.AddToDeck(su1);

            Card su2 = new Card();
            su2.setAll(2, 1);
            aPlayer.AddToDeck(su2);
            aPlayer.AddToDeck(su2);
            aPlayer.AddToDeck(su2);

            Card su3 = new Card();
            su3.setAll(3, 1);
            aPlayer.AddToDeck(su3);

            Card su4 = new Card();
            su4.setAll(4, 1);
            aPlayer.AddToDeck(su4);

            Card su5 = new Card();
            su5.setAll(5, 1);
            aPlayer.AddToDeck(su5);
            aPlayer.AddToDeck(su5);
            aPlayer.AddToDeck(su5);

            Card su6 = new Card();
            su6.setAll(6, 1);
            aPlayer.AddToDeck(su6);
            aPlayer.AddToDeck(su6);

            Card su7 = new Card();
            su7.setAll(7, 1);
            aPlayer.AddToDeck(su7);
            aPlayer.AddToDeck(su7);
            aPlayer.AddToDeck(su7);

            Card su8 = new Card();
            su8.setAll(8, 1);
```

```
aPlayer.AddToDeck(su8);
aPlayer.AddToDeck(su8);
aPlayer.AddToDeck(su8);

Card su9 = new Card();
su9.setAll(9, 1);
aPlayer.AddToDeck(su9);
aPlayer.AddToDeck(su9);
aPlayer.AddToDeck(su9);

Card su10 = new Card();
su10.setAll(10, 1);
aPlayer.AddToDeck(su10);
aPlayer.AddToDeck(su10);
aPlayer.AddToDeck(su10);

Card su11 = new Card();
su11.setAll(11, 1);
aPlayer.AddToDeck(su11);
aPlayer.AddToDeck(su11);
aPlayer.AddToDeck(su11);

Card su12 = new Card();
su12.setAll(12, 1);
aPlayer.AddToDeck(su12);
aPlayer.AddToDeck(su12);

Card su13 = new Card();
su13.setAll(13, 1);
aPlayer.AddToDeck(su13);

Card su14 = new Card();
su14.setAll(14, 1);
aPlayer.AddToDeck(su14);
aPlayer.AddToDeck(su14);
aPlayer.AddToDeck(su14);

Card su15 = new Card();
su15.setAll(15, 1);
aPlayer.AddToDeck(su15);
aPlayer.AddToDeck(su15);
aPlayer.AddToDeck(su15);

Card su16 = new Card();
su16.setAll(16, 1);
aPlayer.AddToDeck(su16);
aPlayer.AddToDeck(su16);
aPlayer.AddToDeck(su16);

Card su17 = new Card();
su17.setAll(17, 1);
aPlayer.AddToDeck(su17);
aPlayer.AddToDeck(su17);

Card su18 = new Card();
su18.setAll(18, 1);
aPlayer.AddToDeck(su18);
```

```
                Card su19 = new Card();
                su19.setAll(19, 1);
                aPlayer.AddToDeck(su19);

                Card su20 = new Card();
                su20.setAll(20, 1);
                aPlayer.AddToDeck(su20);
                aPlayer.AddToDeck(su20);
                aPlayer.AddToDeck(su20);

                Card su21 = new Card();
                su21.setAll(21, 1);
                aPlayer.AddToDeck(su21);
                aPlayer.AddToDeck(su21);
                aPlayer.AddToDeck(su21);

                Card su22 = new Card();
                su22.setAll(22, 1);
                aPlayer.AddToDeck(su22);
                aPlayer.AddToDeck(su22);

                Card su23 = new Card();
                su23.setAll(23, 1);
                aPlayer.AddToDeck(su23);

                aPlayer.Shuffle();

                for (int i = 0; i < 5; i++) aPlayer.DrawACard();
            }
        }
        #endregion
    }
}
```

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace SeniorProject
{
    class Player
    {
        int health;
        List<Card> Deck;
        List<Card> Hand;
        List<Card> Markers;
        int level;

        public Player()
        {
            health = 40; //health starts at 40
            level = 1;
            Deck = new List<Card>();
            Hand = new List<Card>();
            Markers = new List<Card>();
```

```
}

#region Health Functions
public string getHealth()
{
    return health.ToString();
}

public int getHealthAsInt()
{
    return health;
}

public void LoseHealth(int amountLost)
{
    health = health - amountLost;
}

public void GainHealth(int amountGained)
{
    health = health + amountGained;
}

public void setHealth(int newHealth)
{
    health = newHealth;
}
#endregion

#region Get Card Functions
public Card getFromHand(int slot)
{
    return Hand[slot];
}
public Card getFromMarkers(int slot)
{
        return Markers[slot];
}
#endregion

#region Get All Markers
public List<Card> GetAllMarkers()
{
    return Markers;
}

#endregion

#region Add Card Functions
//Place at back of deck/hand/markers when a card is played.
public void AddToHand(Card aCard)
{
    /* If for some reason a card is added that shouldn't be, the first card
     * in Hand is discarded.*/
    if (Hand.Count() >= 5) Hand.RemoveAt(0);
    Hand.Add(aCard);
}
public void AddToDeck(Card aCard)
```

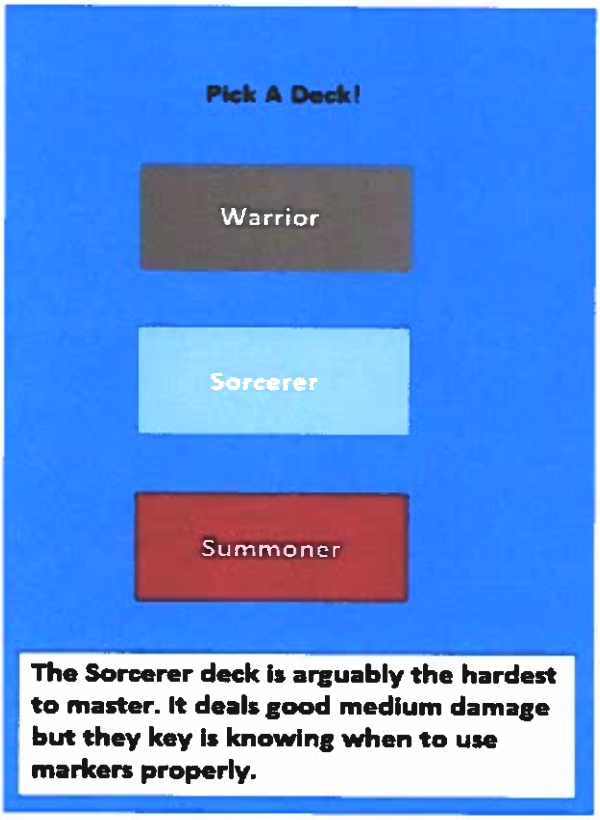
```
        {
            Deck.Add(aCard);
        }
        public void AddToMarkers(Card aCard)
        {
            /* Check number of markers, convention states if markers are more than 3,
then the
             * one in the first slot is removed and the new marker is placed at the end,
             * uses a first-in-first-out functionality.*/
            Markers.Add(aCard);
        }
        #endregion

        #region Remove Card Functions
        public void RemoveHand(Card aCard)
        {
            //Given a card, remove that card from the hand
            Hand.Remove(aCard);
        }
        public void RemoveDeck(Card aCard)
        {
            //If for any reason we need to remove a specific card from deck
            Deck.Remove(aCard);
        }
        public void RemoveMarker()
        {
            if (Markers.Count() > 0)
                Markers.RemoveAt(0);
        }
        public void RemoveMarkerAt(int where)
        {
            if(Markers.Count() > 0)
                Markers.RemoveAt(where);
        }
        #endregion

        #region Draw Function
        ///<summary>
        /// Explanation of list operations:
        /// When Card is drawn, it will be taken from the
        /// Front of the Deck list. When a card is removed,
        /// it uses a function that "removes the first instance"
        /// of a Card inside of the Deck, which is just an easy
        /// way to remove the first item, if you know what that
        /// item is. first-in-first-out is this games convention.
        /// </summary>
        public void DrawACard()
        {
            //Takes the first element in the deck and puts it in hand
            AddToHand(Deck[0]);
            //Removes the item just placed in hand from deck (fist occurrence)
            Deck.RemoveAt(0);
        }
        #endregion

        #region Get Marker Count
        public int getMarkerCount()
        {
```

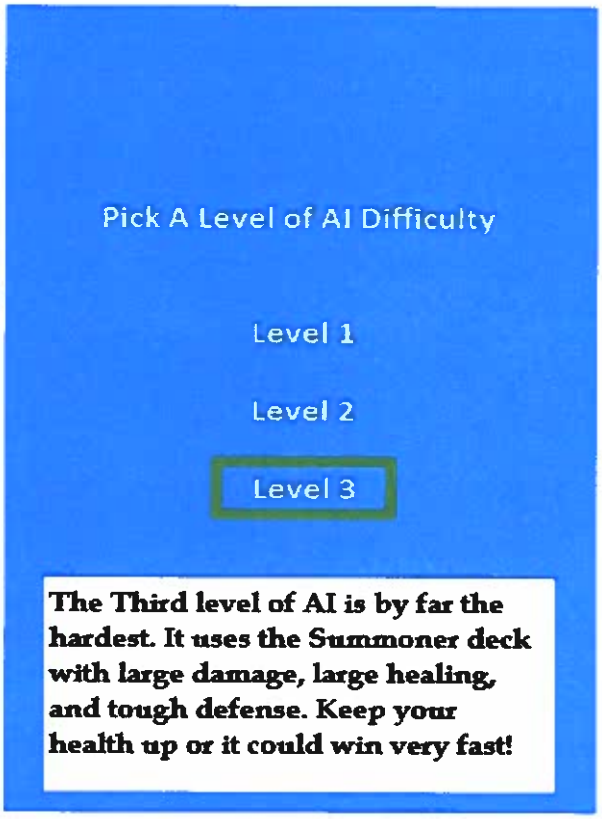
```
            return Markers.Count();
        }
        #endregion

        #region Is Deck Empty?
        public bool DeckEmpty()
        {
            if (Deck.Count() == 0) return true;
            else return false;
        }
        #endregion

        #region Shuffle
        public void Shuffle()
        {
            //Best-known random shuffle using Fisher-Yates implementation*
            int n = Deck.Count;
            Random rnd = new Random();
            while (n > 1)
            {
                int k = (rnd.Next(0, n) % n);
                n--;
                Card value = Deck[k];
                Deck[k] = Deck[n];
                Deck[n] = value;
            }
        }
        #endregion

        //For AI only:
        #region AI Functions
        public void setLevel(int l)
        {
            level = l; //ONLY for future versions! allows for specifics for
summoner/sorcerer/warrior
        }

        public Card AILevel1()
        {
            //This is still reasonably challenging... It's not always so easy.
            return Hand[0];
        }

        public Card AILevel2(Card CurrentCard, int humanHealth, int markerCount)
        {
            int topDamage = 0;
            int topHeal = 0;
            int highestDamagecard = 0;
            int highestHealcard = 0;
            //always win if human will die
            for (int i = 0; i < 5; i++)
            {
                if (Hand[i].getDamageThem() > topDamage)
                {
                    topDamage = Hand[i].getDamageThem();
                    highestDamagecard = i;
                }
            }
```

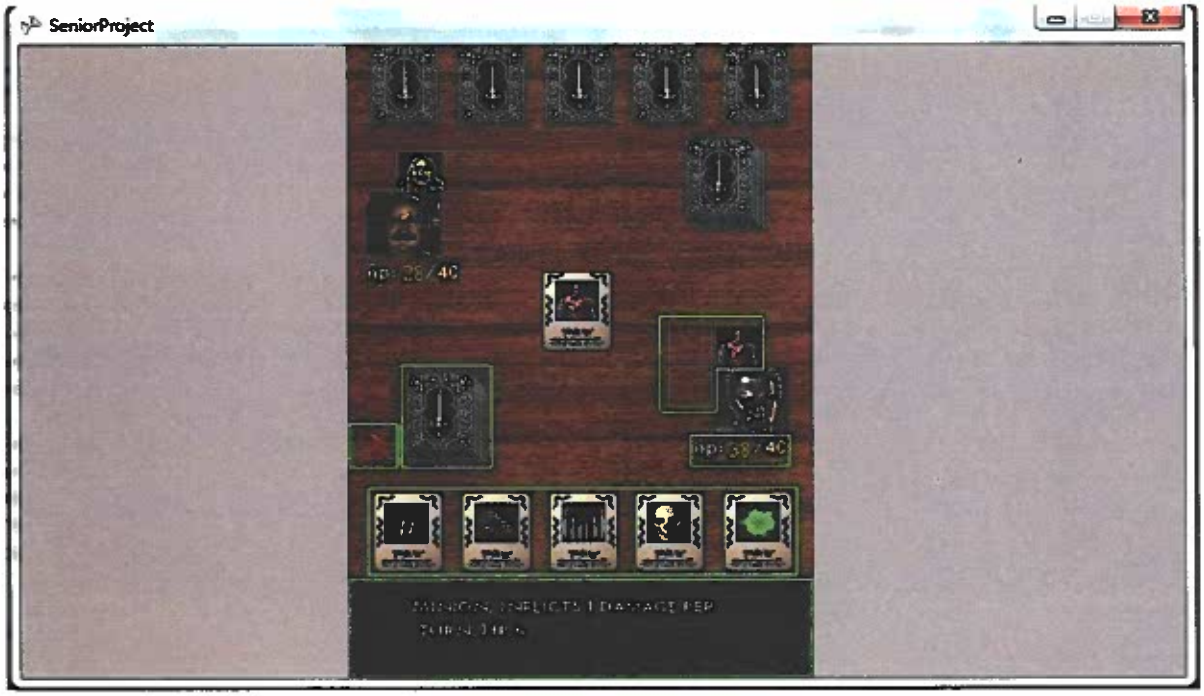
```
            if (humanHealth < topDamage)
            {
                return Hand[highestDamagecard];
            }

            //always heal if there is a chance of dying
            if (health < 10)
            {
                for (int i = 0; i < 5; i++)
                {
                    if (Hand[i].getHeal() > topHeal)
                    {
                        topHeal = Hand[i].getHeal();
                        highestHealcard = i;
                    }
                }
                if (topHeal > 0)
                {
                    return Hand[highestHealcard];
                } //sometimes we don't have a healing card
            }

            //before we return the highest damage card, level 2 will actually want to
            //play a destroy marker card if there exists one in hand and player has
            //2 or more markers... This has been found to be a good strategy, since
            //the sorcerer advantage is to destroy markers
            for (int i = 0; i < 5; i++)
            {
                if (Hand[i].SeeIfRemovesMarker())
                {
                    if (markerCount > 1)
                    {
                        return Hand[i];
                    }
                }
            }

            return Hand[highestDamagecard]; //if nothing else, the highest damage card is
usually a good pick
            //if there are no "damage" cards, this will pick the last card in hand, which
is random enough.
        }

        public Card AILevel3(Card CurrentCard, int humanHealth)
        {
            //use same strategy as level 2, but with summoner deck we have much bigger
advantages
            int topDamage = 0;
            int topHeal = 0;
            int highestDamagecard = 0;
            int highestHealcard = 0;
            int preferMinion = 7; //this will make sense later
            for (int i = 0; i < 5; i++)
            {
                if (Hand[i].SeeIfMinion() == true)  //level3 deck PREFERS MINIONS!
                {
                    preferMinion = i;
                }
```

```
                if (Hand[i].getDamageThem() > topDamage)
                {
                    topDamage = Hand[i].getDamageThem();
                    highestDamagecard = i;
                }
            }
            if (humanHealth < topDamage)
            {
                return Hand[highestDamagecard];
            }
            if (health < 10)
            {
                for (int i = 0; i < 5; i++)
                {
                    if (Hand[i].getHeal() > topHeal)
                    {
                        topHeal = Hand[i].getHeal();
                        highestHealcard = i;
                    }
                }
                if (topHeal > 0)
                {
                    return Hand[highestHealcard];
                }
            }

            if(preferMinion != 7 && health < 35)
                return Hand[preferMinion]; //right here, if neither human or AI have low
health, we want
                //to play a minion. UNLESS health > 35, then just look for highest damage
card.
            return Hand[highestDamagecard];
        }

        public Card AITurn(Card CurrentCard, int humanHealth, int HumanMarkerCount)
        {
            if (level == 1)
            {
                return AILevel1();
            }
            else if (level == 2) return AILevel2(CurrentCard, humanHealth,
HumanMarkerCount);
            else return AILevel3(CurrentCard, humanHealth);
        }
        #endregion

    }
}
```

Screenshots






SeniorProject

CD Information

Game CD

This CD includes the full project, inside of the "Senior Project" folder. If Microsoft Visual Studio is usable on the machine that the CD runs on, then the project may be opened via "SeniorProject.sln" which is a visual studio file. This project is limited to running on only Microsoft (windows) machines. It requires a mouse/mousepad to run properly. If Visual Studio is not on the machine that the CD is on, then there is an alternative executable file. This is located in \SeniorProject\SeniorProject\SeniorProject\bin\x86\Debug. It is called "SeniorProject.exe."

Research and Credits

Sources

http://www.freesound.org/people/Erokia/packs/14615/

http://www.gamedev.net/topic/544073-free-2d3d-art-assets-09/

http://rbwhitaker.wikidot.com/xna-tutorials

http://www.reinerstilesets.de/2d-grafiken/2d-humans/

http://www.spikie.be/blog/page/Building-a-main-menu-and-loading-screens-in-XNA.aspx

http://stackoverflow.com/questions/9712932/2d-xna-game-mouse-clicking

http://xbox.create.msdn.com/en-US/education/catalog/sample/game_state_management

http://www.xnadevelopment.com/tutorials.shtml

http://www.redshift.hu/

http://www.microsoft.com/en-us/download/details.aspx?id=23714